
Security Review Report

NM-0848 - Variational



NETHERMIND
SECURITY

(July 14, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	4
4	System Overview	5
4.1	Omin API server - HTTP and Websocket endpoints	5
4.1.1	HTTPS	6
4.1.2	Websockets	6
4.2	Onchain Transaction Worker	6
4.3	Smart contracts	7
4.4	Onchain Event Handler	7
4.5	Signing API	7
5	Risk Rating Methodology	8
6	Issues	9
6.1	[Critical] Concurrent requests can duplicate referral reward payouts	9
6.2	[Critical] Forged withdrawal failures prevent balance debits	10
6.3	[High] Oracle tracks incomplete list of settlement pools	12
6.4	[High] The batchAtomicDeposit(...) function overcharges the batch creator	13
6.5	[High] Unreachable OLP deposit handler prevents database updates	14
6.6	[Medium] Failed fee collection batches mark transfers as claimed	15
6.7	[Medium] IPv6 clients can bypass per-IP rate limiting	16
6.8	[Medium] Missing authentication and rate limits on the /quote WS endpoints	16
6.9	[Low] Authentication flow trusts Redis records, letting session JWTs be minted for any address	17
6.10	[Low] Cloudflare-only ALB restriction is bypassable by other tenants	19
6.11	[Low] Incomplete is_success() coverage logs successful transactions as failed	20
6.12	[Low] Missing rate limits on unauthenticated endpoints that use pricing service	21
6.13	[Low] No session revocation and no absolute session lifetime	21
6.14	[Low] Potential for double execution if transaction worker crashes	21
6.15	[Low] Redis is deployed without authentication	22
6.16	[Low] The dispatch_direct_deposit(...) function lacks retry logic and duplicate detection	23
6.17	[Info] Account row created before referral validation	23
6.18	[Info] Consumed quotes remain available in cache	24
6.19	[Info] HMAC authentication is not constant-time over key existence	24
6.20	[Info] IP rate limiting fails open when client IP is unresolved	24
6.21	[Info] Identical OLP deposits cannot be reliably distinguished	25
6.22	[Info] Missing company validation on WS re-subscriptions leaks pricing data	25
6.23	[Info] Missing quote UUID zero check allows pulling funds up to max allowance	26
6.24	[Info] No binding between the authenticated caller and the signing key it may use	26
6.25	[Info] No onchain fee segregation for SettlementPools	27
6.26	[Info] Referral capacity check is a TOCTOU race	27
6.27	[Info] Self role removal via renounceRole(...) causes provider counter desync	27
6.28	[Info] Signature recovery id is not constrained to 0, 1	28
6.29	[Info] Terraform CI IAM user has effective admin privileges and escalation paths	28
6.30	[Info] The claimed_at timestamp is never updated due to conflicting SQL logic	29
6.31	[Info] grantRole(...) cannot administer the PROVIDER_ROLE correctly	30
6.32	[Info] post_limit_order(...) skips validation and accepts non-positive qty	30
6.33	[Info] The function permit_helper does not support Safe or ERC-4337 wallets	31
6.34	[Info] The price endpoint lacks a total subscription cap per connection	32
6.35	[Info] Unnecessary USDC double transfers	33
6.36	[Info] WebSocket rate limit result is ignored in /prices	33
6.37	[Info] Lazy address parsing can unexpectedly abort the process	34
6.38	[Best Practices] Decrypted SSM HMAC secret is included in error messages	34
6.39	[Best Practices] Error handling recommendations	34
6.40	[Best Practices] Fragmented fee collection	35
6.41	[Best Practices] Lack of role segregation between standard withdrawals and treasury operations	35
6.42	[Best Practices] Missing functions in IOracle or parent interfaces	36
6.43	[Best Practices] Missing input validation on referral code in set_code(...)	36
6.44	[Best Practices] The SettlementPoolDeployer contract and interface is unused	37
6.45	[Best Practices] Unused dependencies are left in production contract	37
7	Documentation Evaluation	38
8	Test Suite Evaluation	39
8.1	Tests Output	39
9	About Nethermind	41

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Variational](#)'s trading and settlement infrastructure comprising of onchain and offchain components. The infrastructure in scope for this audit comprises of the user-facing API for authentication, trading, and account management, the transaction execution pipeline that submits signed operations onchain, a cryptographic signing authority, and the onchain reconciliation engine that synchronizes blockchain outcomes back into the offchain ledger.

- **Omni API Server:** The primary user-facing interface handling SIWE-based authentication, JWT session management, settlement pool provisioning, USDC deposits/withdrawals via EIP-2612 permits, quote fetching, order execution (market, limit, stop-loss, take-profit, close-all) with margin validation, and real-time WebSocket streaming of positions, prices, and events. Also has ancillary features for User interfaces.
- **On-Chain Transaction Worker:** Bridges the off-chain database and Arbitrum by polling a priority-ordered transaction queue, verifying ERC-20 allowance before deposits, routing pool-scoped operations through deterministic hash-based worker assignment for nonce ordering, and rolling back off-chain state on failure.
- **Signing API:** The centralized signing authority ensuring private keys never leave AWS KMS, authenticating every request via HMAC, and validating each transaction against a strict allowlist of permitted sender-contract-method combinations before signing.
- **On-Chain Event Handler:** Reconciles confirmed on-chain outcomes into the off-chain database by subscribing to Arbitrum blocks with configurable confirmation depth, parsing protocol contract transactions, and applying state transitions (crediting deposits, finalizing withdrawals, opening pools, rolling back failures).

The audit comprises **1120** lines of Solidity code and **25619** lines of Rust code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 43 points of attention, where 2 are classified as Critical, 3 are classified as High, 3 are classified as Medium, 8 are classified as Low and 29 are classified as Informational or Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

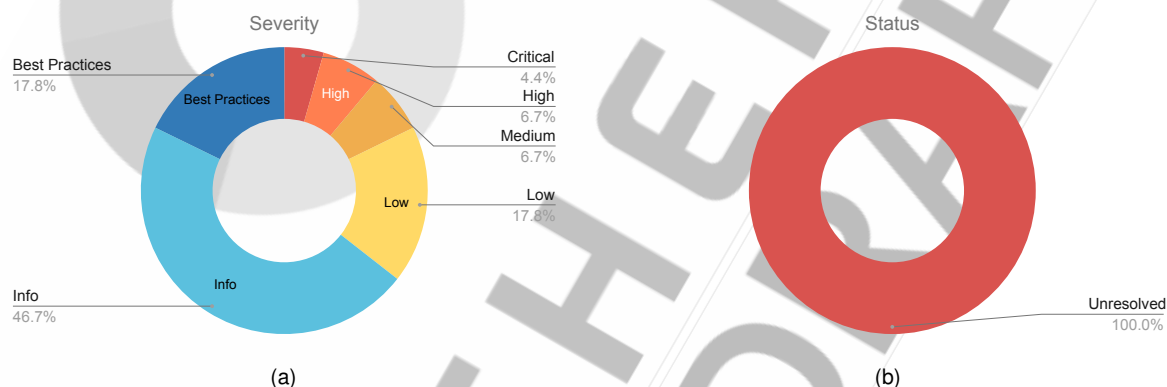


Fig. 1: Distribution of issues: Critical (2), High (3), Medium (3), Low (8), Undetermined (0), Informational (21), Best Practices (8). Distribution of status: Fixed (0), Acknowledged (0), Mitigated (0), Unresolved (45)

Summary of the Audit

Audit Type	Security Review
Initial Report	July 14, 2026
Final Report	-
Initial Commit Infrastructure	a8a8b6921e87fe953270e92803e2b525605d8650
Initial Commit Smart Contracts	b4e316f0998d27fe56da646149b38cc482b05b0d
Final Commit Infrastructure	-
Final Commit Smart Contracts	-
Documentation Assessment	High
Test Suite Assessment	Medium

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	crates/signing_api/src/config.rs	20	4	20.0%	4	28
2	crates/signing_api/src/auth.rs	126	5	4.0%	24	155
3	crates/signing_api/src/metrics.rs	49	4	8.2%	5	58
4	crates/signing_api/src/parse.rs	73	4	5.5%	8	85
5	crates/signing_api/src/signer.rs	232	13	5.6%	41	286
6	crates/signing_api/src/main.rs	123	14	11.4%	23	160
7	crates/signing_api/src/http.rs	164	18	11.0%	17	199
8	crates/signing_api/src/validate.rs	139	2	1.4%	7	148
9	crates/crons/src/on_chain_tx_worker/worker.rs	174	10	5.7%	8	192
10	crates/crons/src/on_chain_tx_worker/metrics.rs	134	1	0.7%	7	142
11	crates/crons/src/on_chain_tx_worker/nonce.rs	164	19	11.6%	15	198
12	crates/crons/src/on_chain_tx_worker/balance.rs	61	4	6.6%	4	69
13	crates/crons/src/on_chain_tx_worker/calls.rs	1082	46	4.3%	54	1182
14	crates/crons/src/on_chain_tx_worker/mod.rs	749	33	4.4%	66	848
15	crates/crons/src/on_chain_tx_worker/main.rs	74	6	8.1%	11	91
16	crates/crons/src/on_chain_tx_worker/allowance.rs	216	15	6.9%	16	247
17	crates/crons/src/on_chain_event_handler/block_tracker.rs	141	17	12.1%	20	178
18	crates/crons/src/on_chain_event_handler/handlers.rs	997	28	2.8%	50	1075
19	crates/crons/src/on_chain_event_handler/events.rs	393	8	2.0%	25	426
20	crates/crons/src/on_chain_event_handler/config.rs	40	0	0.0%	6	46
21	crates/crons/src/on_chain_event_handler/metrics.rs	282	10	3.5%	14	306
22	crates/crons/src/on_chain_event_handler/mod.rs	381	22	5.8%	35	438
23	crates/crons/src/on_chain_event_handler/verify.rs	614	15	2.4%	28	657
24	crates/crons/src/on_chain_event_handler/errors.rs	27	3	11.1%	4	34
25	crates/crons/src/on_chain_event_handler/main.rs	117	3	2.6%	8	128
26	crates/crons/src/on_chain_event_handler/datasource.rs	330	28	8.5%	25	383
27	crates/omni/tests/price_stream_publisher_test.rs	392	95	24.2%	97	584
28	crates/omni/src/error.rs	366	53	14.5%	46	465
29	crates/omni/src/config.rs	370	37	10.0%	57	464
30	crates/omni/src/lib.rs	58	1	1.7%	6	65
31	crates/omni/src/portfolio_snapshot.rs	398	224	56.3%	83	705
32	crates/omni/src/leverage_limits_cache.rs	82	5	6.1%	9	96
33	crates/omni/src/metrics.rs	32	3	9.4%	4	39
34	crates/omni/src/order_batch.rs	631	39	6.2%	62	732
35	crates/omni/src/main.rs	159	14	8.8%	20	193
36	crates/omni/src/redis_client.rs	486	59	12.1%	65	610
37	crates/omni/src/http/orders.rs	2754	288	10.5%	264	3306
38	crates/omni/src/http/rfq_helpers.rs	203	4	2.0%	24	231
39	crates/omni/src/http/loss_refund.rs	228	20	8.8%	19	267
40	crates/omni/src/http/positions.rs	351	59	16.8%	38	448
41	crates/omni/src/http/competition.rs	545	30	5.5%	46	621
42	crates/omni/src/http/maintenance.rs	76	2	2.6%	7	85
43	crates/omni/src/http/on_chain.rs	384	26	6.8%	34	444
44	crates/omni/src/http/metadata.rs	299	17	5.7%	38	354
45	crates/omni/src/http/pooints.rs	140	18	12.9%	16	174
46	crates/omni/src/http/transfers.rs	284	35	12.3%	24	343
47	crates/omni/src/http/settlement_pools.rs	565	34	6.0%	40	639
48	crates/omni/src/http/leaderboard.rs	776	58	7.5%	58	892
49	crates/omni/src/http/exports.rs	265	47	17.7%	36	348
50	crates/omni/src/http/candles.rs	241	35	14.5%	29	305
51	crates/omni/src/http/auth.rs	958	139	14.5%	115	1212
52	crates/omni/src/http/funding.rs	71	5	7.0%	10	86
53	crates/omni/src/http/referrals.rs	450	31	6.9%	46	527
54	crates/omni/src/http/portfolio.rs	219	24	11.0%	17	260
55	crates/omni/src/http/mod.rs	785	69	8.8%	83	937
56	crates/omni/src/http/status.rs	34	5	14.7%	6	45
57	crates/omni/src/http/quotes.rs	2233	104	4.7%	174	2511
58	crates/omni/src/http/rate_limit.rs	166	7	4.2%	25	198
59	crates/omni/src/http/trades.rs	122	18	14.8%	12	152
60	crates/omni/src/http/markets.rs	122	11	9.0%	13	146
61	crates/omni/src/http/vault.rs	282	16	5.7%	21	319
62	crates/omni/src/ws/quote_publisher.rs	66	12	18.2%	9	87
63	crates/omni/src/ws/config.rs	121	118	97.5%	21	260
64	crates/omni/src/ws/price_stream_publisher.rs	165	163	98.8%	30	358
65	crates/omni/src/ws/metrics.rs	313	24	7.7%	10	347
66	crates/omni/src/ws/connection_logger.rs	474	124	26.2%	75	673
67	crates/omni/src/ws/mod.rs	1951	280	14.4%	185	2416
68	crates/omni/src/ws/main.rs	100	7	7.0%	10	117
	Total	25619	2692	10.5%	2509	30820

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/TreasuryManagementHelper.sol	85	1	1.2%	20	106
2	src/SettlementPool.sol	283	22	7.8%	27	332
3	src/Oracle.sol	378	43	11.4%	47	468
4	src/SettlementPoolDeployer.sol	38	7	18.4%	5	50
5	src/SettlementPoolFactory.sol	89	9	10.1%	14	112
6	src/library/TreasuryManagement.sol	13	1	7.7%	1	15
7	src/library/Fees.sol	8	1	12.5%	1	10
8	src/library/Errors.sol	7	1	14.3%	1	9
9	src/library/SettlementPools.sol	9	1	11.1%	1	11
10	src/interfaces/ISettlementPoolFactory.sol	30	26	86.7%	8	64
11	src/interfaces/ISettlementPool.sol	9	7	77.8%	2	18
12	src/interfaces/IOracle.sol	11	6	54.5%	2	19
13	src/interfaces/ISettlementPoolDeployer.sol	12	11	91.7%	1	24
14	src/interfaces/oracle/IOracleActions.sol	62	1	1.6%	11	74
15	src/interfaces/oracle/IOracleEvents.sol	36	19	52.8%	8	63
16	src/interfaces/oracle/IOracleOwnerActions.sol	8	4	50.0%	5	17
17	src/interfaces/pool/ISettlementPoolMembers.sol	7	9	128.6%	4	20
18	src/interfaces/pool/ISettlementPoolEvents.sol	35	29	82.9%	7	71
	Total	1120	198	17.7%	165	1483

3 Summary of Issues

	Finding	Severity	Update
1	Concurrent requests can duplicate referral reward payouts	Critical	Unresolved
2	Forged withdrawal failures prevent balance debits	Critical	Unresolved
3	Oracle tracks incomplete list of settlement pools	High	Unresolved
4	The batchAtomicDeposit(...) function overcharges the batch creator	High	Unresolved
5	Unreachable OLP deposit handler prevents database updates	High	Unresolved
6	Failed fee collection batches mark transfers as claimed	Medium	Unresolved
7	IPv6 clients can bypass per-IP rate limiting	Medium	Unresolved
8	Missing authentication and rate limits on the /quote WS endpoints	Medium	Unresolved
9	Authentication flow trusts Redis records, letting session JWTs be minted for any address	Low	Unresolved
10	Cloudflare-only ALB restriction is bypassable by other tenants	Low	Unresolved
11	Incomplete is_success() coverage logs successful transactions as failed	Low	Unresolved
12	Missing rate limits on unauthenticated endpoints that use pricing service	Low	Unresolved
13	No session revocation and no absolute session lifetime	Low	Unresolved
14	Potential for double execution if transaction worker crashes	Low	Unresolved
15	Redis is deployed without authentication	Low	Unresolved
16	The dispatch_direct_deposit(...) function lacks retry logic and duplicate detection	Low	Unresolved
17	Account row created before referral validation	Info	Unresolved
18	Consumed quotes remain available in cache	Info	Unresolved
19	HMAC authentication is not constant-time over key existence	Info	Unresolved
20	IP rate limiting fails open when client IP is unresolved	Info	Unresolved
21	Identical OLP deposits cannot be reliably distinguished	Info	Unresolved
22	Missing company validation on WS re-subscriptions leaks pricing data	Info	Unresolved
23	Missing quote UUID zero check allows pulling funds up to max allowance	Info	Unresolved
24	No binding between the authenticated caller and the signing key it may use	Info	Unresolved
25	No onchain fee segregation for SettlementPools	Info	Unresolved
26	Referral capacity check is a TOCTOU race	Info	Unresolved
27	Self role removal via renounceRole(...) causes provider counter desync	Info	Unresolved
28	Signature recovery id is not constrained to 0, 1	Info	Unresolved
29	Terraform CI IAM user has effective admin privileges and escalation paths	Info	Unresolved
30	The claimed_at timestamp is never updated due to conflicting SQL logic	Info	Unresolved
31	grantRole(...) cannot administer the PROVIDER_ROLE correctly	Info	Unresolved
32	post_limit_order(...) skips validation and accepts non-positive qty	Info	Unresolved
33	The function permit_helper does not support Safe or ERC-4337 wallets	Info	Unresolved
34	The price endpoint lacks a total subscription cap per connection	Info	Unresolved
35	Unnecessary USDC double transfers	Info	Unresolved
36	WebSocket rate limit result is ignored in /prices	Info	Unresolved
37	[Lazy address parsing can unexpectedly abort the process	Info	Unresolved
38	Decrypted SSM HMAC secret is included in error messages	Best Practices	Unresolved
39	Error handling recommendations	Best Practices	Unresolved
40	Fragmented fee collection	Best Practices	Unresolved
41	Lack of role segregation between standard withdrawals and treasury operations	Best Practices	Unresolved
42	Missing functions in IOracle or parent interfaces	Best Practices	Unresolved
43	Missing input validation on referral code in set_code(...)	Best Practices	Unresolved
44	The SettlementPoolDeployer contract and interface is unused	Best Practices	Unresolved
45	Unused dependencies are left in production contract	Best Practices	Unresolved

4 System Overview

Variational is a decentralized perpetual futures trading platform built on Arbitrum blockchain. The audit scope encompasses four interconnected components that together form a system with user facing interfacing, offchain accounting and onchain settlement.

Variational

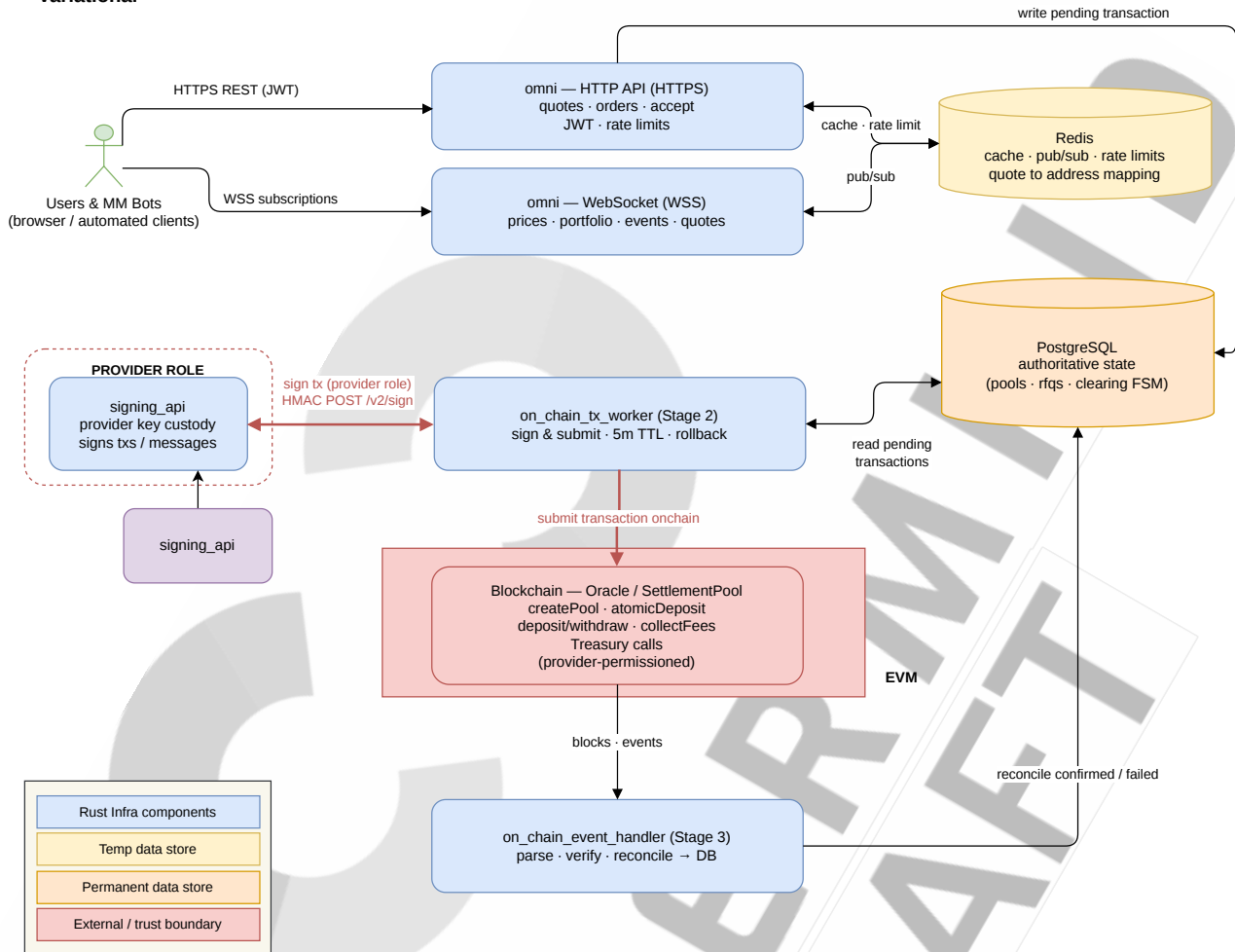


Fig. 2: Variational Perps Trading system overview

4.1 Omin API server - HTTP and Websocket endpoints

The **Omni** API server is the primary entry point for all user interactions. It exposes both HTTP and WebSocket endpoints through which users authenticate using sign-in to receive JWT tokens. The authenticated users can submit orders, request quotes from the OLP quoting engine, manage positions and portfolios, and initiate deposits and withdrawals. Authentication is enforced per-handler via JWT extraction, with multi-tier rate limiting applied at the IP, per-company, and global levels. The WebSocket server runs as a separate component and streams real-time data related to order lifecycle, portfolio updates, prices, and live quotes to authenticated clients.

When a user initiates an onchain action, such as depositing USDC or withdrawing funds, the request does not go directly to the blockchain. Instead, it is queued into the on chain transaction queue table in the PostgreSQL database. The onchain transaction worker component reads these pending records from the table, assigns each transaction to the appropriate signer worker based on transaction type, and submits the transaction onchain. The Oracle smart contract is the main interfacing contract, and functionality is gated to the provider role.

This queued architecture allows the system to batch, prioritize (high/OLP/low), and manage nonces across multiple signing wallets while maintaining deterministic per-pool ordering.

All transaction signing is delegated to the signing API microservice. Every request to this service is HMAC-authenticated (SHA-256 over key, timestamp, method, URI, and body), and the transaction bytes are decoded and validated against a strict allowlist of permitted (sender, contract, method) combinations before the KMS signing call is made. Only EIP-1559 transactions targeting the three known contracts (Oracle, USDC, TreasuryManagementHelper) are accepted.

The record in the database is marked as submitted once the transaction is submitted onchain. To close the loop, the onchain event handler cronjob reads the onchain data block by block. The handler subscribes to new Arbitrum blocks via WebSocket, fetches full block contents, and parses each transaction for relevant contract events.

When it identifies a confirmed deposit, withdrawal, pool creation, atomic deposit, fee collection, or treasury management operation, it updates the offchain database state accordingly, confirming pending transfers, opening settlement pools, updating asset balances, and advancing clearing statuses. The handler persists its block cursor to both Redis and PostgreSQL for crash recovery.

These four components form a closed loop, with user actions flowing through the Omni API into the transaction queue. The transaction queue is processed by a transaction worker that reads the queue records and submits them onchain via the signing API.

The event handler is responsible for updating the transaction status in the database by reading the blocks on the chain.

The Omni crate is the primary user-facing API server for the Variational exchange. It provides an Axum-based HTTP server handling authenticated trading operations and a separate TCP-based Web-socket server for real-time streaming. The HTTP server applies a global middleware stack (CORS, Prometheus metrics, tracing, panic catching) and injects shared dependencies (database pools, pricing clients, OLP quoter, Redis, rate limiters) as Axum extensions. Authentication is per-handler using JWT tokens.

4.1.1 HTTPS

- **Authentication:** Manages the full SIWE-based login flow, JWT issuance and refresh, and a three-step mobile authentication transfer protocol. On first login, it performs a GoPlus security check on the wallet address and creates the user's settlement pool.
- **Quotes:** Fetches indicative and firm quotes, computes margin requirements, and accepts quotes.
- **Orders:** Supports market and limit order submission, order cancellation, and closing all positions.
- **Onchain operations:** Generates and submits EIP-2612 permits.
- **Transfers:** Initiates deposits and withdrawals and provides access to transfer history.
- **Positions:** Allows authenticated users to view the current trading positions in their settlement pool.
- **Portfolio:** Provides authenticated users with an aggregated view of their account health and trading activity.
- **Settlement pools:** Manages the pools underpinning all trading activity on Variational.
- **Trades:** Returns the authenticated caller's paginated trade history.
- **Referrals:** Implements the platform's referral incentive program, allowing users to earn rewards by bringing new traders.

Additional endpoints are available for consumption by the front end:

- **Status:** Returns the server timestamp and authenticated user information.
- **Metadata:** Provides instrument metadata, open interest, protocol configuration, and fee tiers.
- **Candles:** Returns candlestick data for perpetual assets.
- **Funding:** Provides perpetual funding rate information. Unauthenticated.
- **Markets:** Returns seven-day sparkline price arrays at one-hour intervals. Unauthenticated.
- **Leaderboard:** Provides leaderboard data and supports setting display names. Authentication is optional.
- **Competition:** Returns competition results and allows users to join competitions.
- **Points:** Provides points and rewards summaries and history. JWT authentication is required.
- **Loss Refund:** Returns loss refund eligibility and history.
- **Exports:** Supports asynchronous CSV export job creation and status polling. JWT authentication is required.
- **Maintenance:** Returns maintenance mode flags and performs administrator checks. Unauthenticated.

4.1.2 Websockets

The WebSocket server runs as a separate instance. Connections are dispatched by URL path from the raw HTTP upgrade request. Authentication (where required) is performed by expecting a JWT as the first WebSocket message within a 60-second timeout.

4.2 Onchain Transaction Worker

A background service that acts as a bridge between the system's internal database and the blockchain. It constantly monitors a database table for pending transactions. It sends waiting transactions to the correct wallet for signing and posting to the blockchain. The signing is delegated to the Signing API microservice to produce signatures for the transactions. It automatically calculates gas fees, tracks transaction order (nonces), and double-checks token permissions before making deposits.

4.3 Smart contracts

Variational's onchain layer is deployed on Arbitrum utilizes USDC as its sole collateral asset, orchestrated by a central oracle contract that manages all privileged, fund-moving operations initiated by authorized offchain provider nodes. This oracle interfaces with the SettlementPoolFactory to deploy new pools and coordinates directly with individual SettlementPool instances to process USDC in-flows and outflows. The TreasuryManagementHelper serves as a parallel entry point dedicated specifically to executing OLP-funded batch deposit operations.

- **Oracle:**
 - Central orchestrator managing user asset movements, atomic two-party settlements, fee collection, and treasury routing.
 - Implements OpenZeppelin guards, administrative emergency controls, and restricted to authorized PROVIDER_ROLE access.
 - Implements transaction replay protection
- **SettlementPoolFactory:**
 - Deploys lightweight bilateral pool instances using the EIP-1167 minimal proxy pattern pointing to a single logic contract.
 - Maintains the canonical UUID-to-address pool registry alongside the immutable USDC collateral token address.
 - Exposes createPool to the owner or oracle to clone the implementation, initialize parties, and optionally collect creation fees.
- **SettlementPool:**
 - Bilateral escrow proxy holding USDC collateral for exactly two sides, initialized once by the factory instead of a constructor.
 - Restricts access via onlyParties for direct deposits and onlyOracle for automated, dynamically resolved oracle operations.
 - Implements replay protection across standard, atomic, fee extraction, and eventless treasury transfers.
- **TreasuryManagementHelper:**
 - Acts as a dedicated secondary entry point optimized specifically for executing batch deposit operations funded by the OLP.
 - Governed by ORACLE_PROVIDER_ROLE to securely pull total USDC deposit amounts from the OLP wallet.

4.4 Onchain Event Handler

The Onchain Event Handler acts as the system's reconciler between onchain and offchain datasets. It listens to the blockchain, reads transaction data, and updates the internal PostgreSQL database accordingly. The main loop runs continuously, processing multiple transactions simultaneously to improve speed. A watchdog system will completely shut down the service if no new blocks arrive or if a block takes longer than 60 seconds to process. It also features a manual mode that allows replaying a single transaction by its hash.

4.5 Signing API

The signing API is a standalone microservice responsible for all transaction signing across the platform. In production, private keys are managed exclusively within AWS KMS and never leave the hardware security module; in development, keys are loaded from environment variables into process memory. The service exposes a single HMAC authenticated signing endpoint and an unauthenticated status endpoint.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Critical] Concurrent requests can duplicate referral reward payouts

File(s): crates/database/src/referrals.rs

Description: Companies accrue referral rewards, stored as reward rows marked available. A company withdraws them by calling POST /referrals/claim_rewards. The endpoint adds up the company's available rewards, creates a transfer for that total (minus a fixed transfer fee), marks those rewards as claimed, and queues an on-chain payment of the amount into the company's settlement pool. The whole operation is a single database query inside claim_available_rewards(...). Neither the request handler nor the query takes any lock or runs in a serializable transaction, so nothing prevents two requests for the same company from executing at the same time. The query has three relevant parts:

```
// crates/database/src/referrals.rs, claim_available_rewards(...)
let new_transfer = sqlx::query_as!(DbTransferRecord, r#"
    WITH available AS (SELECT ARRAY_AGG(id) AS ids, SUM(qty) AS qty
                        FROM referral_reward_claims
                        WHERE company = $1 AND status = 'available'
                        HAVING SUM(qty) > $4),
    claim_transfer AS (INSERT INTO transfers (/* ... */)
                      SELECT $1, qty - $4, 'USDC', /* ... */
                      FROM available WHERE qty > 0
                      RETURNING /* ... */),
    // ...
    // @audit-issue Marks the rewards claimed by id only, without
    // @audit-issue re-checking that they are still 'available'.
    update_claims AS (UPDATE referral_reward_claims
                      SET status = 'pending', transfer_id = /* ... */
                      WHERE id = ANY(available.ids))
"#, company, pool_location, counterparty, fee_amount)
// @audit No lock or serializable transaction guards this query.
.fetch_optional(&self.pg_pool)
.await?;
```

When two claim requests for the same company arrive close together, they run this query at the same time and interfere in two ways. First, both read the list of available rewards and compute the same total before either has finished, so both proceed as if the full balance were still unclaimed. Second, the part that marks the rewards claimed matches them only by their id and never re-checks that they are still available, and the payout amount comes from the total read at the start rather than from the rewards the update actually changed. As a result, the second request still creates a full payout even though the first has already claimed the same rewards. There is also no safety net in the database: nothing ties a reward to a single transfer, and the running available-balance total has no check preventing it from going below zero.

An attacker who controls a company with accrued referral rewards can be paid the same rewards many times over:

1. Let the company hold available rewards totalling Q (greater than the transfer fee);
2. Send many POST /referrals/claim_rewards requests for that company at the same time. The per-company rate limit defaults to 100 per window, so a large burst is allowed;
3. Because none of the requests has committed yet, each one reads the same available rewards and computes the same total Q ;
4. Each request creates a transfer of $Q - \text{fee}$ and queues an on-chain payment for it;
5. All of these payments are sent on-chain into the company's settlement pool. For N simultaneous requests the protocol pays out roughly $N * (Q - \text{fee})$ while only Q was ever earned, and the recorded available balance goes negative;

Nothing later in the pipeline catches the duplicates. Each racing request produces a separate transfer with its own id, so the on-chain payment queue accepts one entry per transfer and the signer worker sends each one independently. The on-chain function used for these payouts, transferFromOLPToPool(...), moves the funds unconditionally and keeps no record of processed transfers, unlike the related atomicDeposit(...) function which rejects an already-processed transfer. Even such a guard would not help here, because it is keyed on the transfer id, which is different for every duplicate. Once sent, the payments cannot be reversed.

The practical impact is amplified by the separate IPv6 rate-limit bypass described elsewhere in this report. In testing, the race was demonstrated with a limited IPv4 request burst, producing an approximately 8x duplicate claim. However, if an attacker can bypass the per-company or per-IP throttling by rotating IPv6 source addresses, they can attempt a much larger concurrent burst. With the default limit allowing up to 100 requests per window, this can plausibly turn the same race from an 8x overpayment into an approximately 100x overpayment. The attack also does not need to be performed immediately when rewards are earned. A malicious company can wait until its available reward balance is large enough to make the race economically significant. For example, testing showed that roughly \$41 of referral rewards could be accrued in only a few days, so accumulating a balance on the order of \$1,000 is feasible. If that \$1,000 balance is then claimed through a successful 100-request race, the protocol could queue and pay roughly \$100,000 from the treasury for rewards that were only earned once.

Recommendation(s): Consider making the read-and-consume atomic against concurrent callers. Adding a database constraint that forbids a negative available balance would also turn any remaining race into a safe failure rather than a silent overpayment.

Status: Unresolved

Update from the client:

6.2 [Critical] Forged withdrawal failures prevent balance debits

File(s): crates/database/src/transfers.rs, crates/crons/src/on_chain_event_handler/handlers.rs

Description: Settlement pools hold pooled USDC collateral for the parties that trade against each other in the pool. When a user requests a withdrawal, the backend inserts a pending row in the transfers table and queues an onchain withdrawUSDC call. The row is written by `insert_pending_transfer_in_tx(...)` in `transfers.rs`, which the backend calls with the status set to Pending:

```
sqlx::query_as!(
    DbTransferRecord,
    r#"
    INSERT INTO public.transfers (
        id, company, counterparty, asset, target_pool_location, qty,
        transfer_type, status
        // ...
    )
    VALUES($1, $2, $3, $4, $5, $6, $7, $8 /* ... */)
    // ...
    "#,
    request.transfer_id,
    // ...
    // @audit The status is Pending; no balance is debited when the row is created.
    request.transfer_status as TransferStatus,
    // ...
)
.fetch_one(&mut **tx)
.await
```

Its caller, `impl_insert_pending_transfer(...)`, then queues the matching onchain call for a Withdrawal in the same database transaction:

```
let transfer = self.insert_pending_transfer_in_tx(&mut tx, &request).await?;
// ...
if insert_onchain_call && matches!(request.transfer_status, TransferStatus::Pending) {
    // ...
    let call = match request.transfer_type {
        // ...
        TransferType::Withdrawal => {
            serde_json::to_value(OracleCalls::withdrawUSDC(Oracle::withdrawUSDCall {
                requestor: owner,
                amountRequested: usdc_to_base_units_rounded_up(request.qty.abs())?,
                poolUuid: pool_location.as_u128(),
                transferUuid: transfer.id.as_u128(),
            }))?
        }
        // ...
    };
    sqlx::query!(
        r#"
        INSERT INTO on_chain_transaction_queue (contract, call, transfer_id)
        VALUES ($1, $2, $3)
        "#,
        QueuedContractType::Oracle as QueuedContractType,
        call,
        transfer.id,
    )
    .execute(&mut *tx)
    .await?;
}
```

The `on_chain_tx_worker` cron later polls the queue and submits that call onchain, signing it with a privileged oracle provider wallet (an address in `ORACLE_PROVIDER_ADDRESSES`, holding the onchain `PROVIDER_ROLE`); the settlement pool then sends USDC from the pool to the requestor. The user's offchain balance in the `assets` table is the sole record of how much they are entitled to withdraw: the onchain pool performs no per-party accounting, and the `_withdrawUSDCOnBehalf(...)` function in `SettlementPool.sol` only checks that the requestor is a pool party and that the `transferUuid` has not been used before, then transfers the requested amount out of the pool's total balance.

The offchain balance is debited only when the onchain withdrawal is confirmed. The event handler observes the mined withdrawUSDC transaction and branches on the outcome in `handle_withdrawal(...)`:

```
match outcome {
  Outcome::SuccessWithEvent(_) => {
    handle_transfer_success(tx, transfer_id, onchain_amount, db_client).await?;
  }
  // @audit-issue The failure path only marks the transfer Failed; no balance change.
  Outcome::Failure => {
    handle_transfer_failure(transfer_id, db_client).await?;
  }
}
```

Only the success path (`handle_transfer_success(...)`) subtracts the amount from the user's balance; the failure path (`handle_transfer_failure(...)`) merely marks the transfer Failed and leaves the balance untouched. A withdrawal request itself reserves nothing. Three properties let a user permanently suppress that debit while still receiving the funds. First, reverted transactions are not discarded. In `events.rs`, `parse_receipt(...)` turns a reverted receipt into a Failure outcome rather than dropping it:

```
fn parse_receipt<C, E: SolEvent>(<
  receipt: &AnyTransactionReceipt,
  call: C,
) -> Option<ParsedReceipt<C, E>> {
  // @audit-issue A reverted transaction is kept as a Failure outcome, not dropped.
  if !receipt.status() {
    return Some(ParsedReceipt {
      call,
      outcome: Outcome::Failure,
      // ...
    });
  }
  // ...
}
```

Second, any transaction sent to the oracle address with `withdrawUSDC` calldata is decoded regardless of its sender (there is no sender restriction on this branch of `parse_transaction(...)` in `events.rs`), so any address can produce such a Failure outcome. Third, the verification in `verify_transfer_event(...)` in `verify.rs` accepts a transaction whose sender equals the transfer's own party address, and only acts on a transfer that is still Pending:

```
// @audit The sender check passes when tx.from() equals the party's own address.
if &tx.from() != company_addr && !ORACLE_PROVIDER_ADDRESSES.contains(&tx.from()) {
  // ...
  return Ok(false);
}
// ...
// @audit-issue Only a Pending transfer is actionable, so the first terminal write wins.
if r.status == TransferStatus::Pending {
  true
} else {
  // ...
  false
}
```

In the failure branch of `handle_withdrawal(...)` the party address passed as `company_addr` is taken from the transaction calldata (`call.requestor`), which the sender controls. The transaction worker in `calls.rs` broadcasts the real withdrawal in `dispatch_withdraw_usdc(...)` without re-reading the transfer's status:

```
async fn dispatch_withdraw_usdc<P: Provider<AnyNetwork>>(<...> -> ResultOrCancel<Option<TxHash>, Error> {
  // ...
  send_detect_dupe(
    // @audit Broadcast without re-reading the transfer's DB status.
    contract.withdrawUSDC(
      withdraw.requestor,
      withdraw.amountRequested,
      withdraw.poolUuid,
      withdraw.transferUuid,
    ),
    // ...
  )
  .await
  // ...
}
```

Because the pool has no per-party accounting, a user can drive their own in-flight withdrawal to Failed before the genuine payout is recorded. An attacker who is a party in an open pool can do the following:

1. Request a legitimate withdrawal of X. The API returns the transferUuid T, and no balance is debited yet;
2. The backend commits a pending transfers row for T and queues the call; the on_chain_tx_worker then submits withdrawUSDC(requestor = attacker, X, pool, T) on-chain (signed by an oracle provider wallet), which succeeds and sends X to the attacker;
3. In parallel, the attacker repeatedly sends their own withdrawUSDC(requestor = attacker, X, pool, T) directly to the oracle. These revert at the PROVIDER_ROLE check (before the on-chain transferUuid dedupe), so no funds move and T is not marked processed on-chain, but each reverted transaction is still mined;
4. The event handler processes the attacker's reverted transaction as Failure. The sender check passes (tx.from() equals call.requestor), the transfer is still Pending, so handle_transfer_failure(...) marks T as Failed with no balance change;
5. Blocks are processed strictly in order (the main loop awaits all handlers for a block before advancing), so once the reverted transaction is committed as Failed in an earlier block, the genuine success in a later block is skipped because the transfer is no longer Pending. The debit never happens;
6. The attacker has received X on-chain but their off-chain balance is unchanged. They repeat with a fresh transferUuid and withdraw X again, paid out from the pool's pooled collateral, which belongs to the counterparties;

The result is theft of other parties' funds and pool insolvency. It also gives the attacker risk-free trading: because the assets table is never debited, the withdrawn X still counts as usable collateral, so the attacker can keep opening positions backed by funds they have already removed from the pool. Any trading losses can no longer be settled against that collateral (it has already left the pool on-chain), while any gains remain withdrawable, so the attacker trades with no real downside at the counterparties' expense. The attack costs only gas on the reverting transactions and is repeatable. The window between the pending row appearing and the genuine success being observed spans the on-chain mining and block-polling latency (seconds), and the attacker knows T from the outset, so the race strongly favors the attacker.

Recommendation(s): Consider treating a Failure outcome as authoritative only when the reverted transaction actually originated from a trusted oracle provider address, rather than from the transfer's own party.

Status: Unresolved

Update from the client:

6.3 [High] Oracle tracks incomplete list of settlement pools

File(s): src/Oracle.sol

Description: When a new settlement pool is created in the SettlementPoolFactory, an entry into the getPool mapping is created to associate the pool UUID with the newly created pool address. This mapping can be queried via a public function and acts as the source of truth for retrieving pools based on their UUID, and pool existence checks. When creating a new pool, the usual flow is to have the offchain Omni infrastructure call Oracle.createPool which then calls SettlementPoolFactory.createPool, at which point the pool is created and the entry into the SettlementPoolsFactory.getPools is done. However, the Oracle contract also has its own storage mapping Oracle.pools to associate pool UUIDs to addresses. When a pool is created through Oracle.createPool an entry is submitted into this mapping.

```
contract Oracle {
    function createPool(...) ... {
        require(pools[poolUuid] == address(0));
        address poolAddress = factory.createPool(...); // Call factory to create pool
        pools[poolUuid] = poolAddress;                 // Update 'pools' mapping
    }
}

contract SettlementPoolFactory {
    function createPool(...) ... returns (address pool) {
        require(msg.sender == owner() || msg.sender == address(oracle));
        // ...
        pool = Clones.clone(implementation);           // Create pool
        SettlementPool(pool).initialize(...);           // Initialize pool
        getPool[poolUuid] = pool;                       // Update 'getPool' mapping
        // ...
    }
}
```

This means we have two contracts with two unique storage mappings which are supposed to track the same data. There are two scenarios where it is possible for the data in both mappings to differ, the first being a direct pool creation and the second being an oracle upgrade. Each scenario will be described below:

1. Direct pool creation: The SettlementPoolFactory.createPool function has a permission gate that allows either the oracle or the owner to create a pool. Under normal use all account creation is done through the oracle, however if a settlement pool is created by having the factory owner call createPool directly, this will skip the flow through the oracle that would have created an entry in Oracle.pools;
2. Oracle upgrade: The SettlementPoolFactory has a function setOracleAddress which will be used to change out to a new oracle implementation. When a new oracle implementation is swapped in, it would have an empty Oracle.pools mapping. This will immediately cause a difference in mapping data, since the new oracle mapping is empty and the factory mapping contains all deployed pools.

The Oracle contract doesn't use the same approach to find a pool address given a UUID. In some functions it will use its own incomplete pools mapping, and other times it will use complete `getPool` from the factory. In particular, the incomplete mapping is used in `withdrawUSDC`, `atomicDeposit`, and `depositUSDC`. When the oracle contract is missing an entry in its mapping for a particular settlement pool, any interaction with the aforementioned functions will revert.

This doesn't block all deposits, there are other functions in the oracle and in the settlement pool itself that allow deposits to continue to operate successfully. For withdrawals however, the only way to withdraw collateral from a pool is via the `Oracle.withdrawUSDC` function. Since this function does a pool address lookup using the incomplete pools mapping it can revert upon a missing entry. This means that deposits can still operate properly (for some flows) but withdrawals will always fail, locking user collateral in the pool. Users may happen to deposit using the flows that still work, and not realize that their account has an issue until they attempt to withdraw.

Recommendation(s): Consider removing the pools mapping and the internal `getPool` function from the Oracle contract, and instead use the factory `getPool` function as the source of truth for all UUID to pool address queries.

Status: Unresolved

Update from the client:

6.4 [High] The `batchAtomicDeposit(...)` function overcharges the batch creator

File(s): `src/Oracle.sol`

Description: The Oracle contract contains the `batchAtomicDeposit(...)` function, which allows the provider role to deposit USDC into a settlement pool for both the creator and multiple other parties (represented in the `items` array) in a single transaction. This function is designed to charge the batch creator a token amount represented by the `creatorPartyAmountRequested` variable:

```
function batchAtomicDeposit(
    ...,
    uint256 creatorPartyAmountRequested,
    SettlementPools.AtomicDepositBatchItem[] calldata items
) ... onlyRole(PROVIDER_ROLE) {
    // ...
    pool.batchDepositUSDCAtomic(
        creatorPartyAddress,
        creatorPartyAmountRequested, // This represents the amount the creator will deposit
        items // This array contains the multiple deposits and amounts of other parties
    );
}
```

When the Oracle contract then calls the relevant settlement pool function `batchDepositUSDCAtomic`, a loop executes which iterates over every item in the batch. The charge for the creator is inside the loop, meaning it is executed n times where n is the number of other parties in the `items` array. As a result, the creator will be overcharged on their deposit by multiples over their expected deposit.

```
function batchDepositUSDCAtomic(...) external nonReentrant onlyOracle {
    for (uint i = 0; i < items.length; i++) {
        // ...
        if (creatorPartyAmountRequested > 0) { // Creator deposit is done on every loop
            _depositUSDCOnBehalf(...);
            // ...
        }
        if (items[i].otherPartyAmountRequested > 0) {
            _depositUSDCOnBehalf(...);
        }
    }
    // ...
}
```

As an example, consider a batch deposit where the creator deposits 100, and there are two other parties depositing 50. The expected total deposit would be $100 + 50 + 50$ however the actual deposit amount would be $100 + 100 + 50 + 50$ since the creator deposit is done multiple times, one for each other party.

Recommendation(s): Consider moving the creator deposit logic outside of the loop so that it is only executed once.

Status: Unresolved

Update from the client:

6.5 [High] Unreachable OLP deposit handler prevents database updates

File(s): crates/crons/src/on_chain_tx_worker/calls.rs, crates/crons/src/on_chain_event_handler/events.rs, crates/crons/src/on_chain_event_handler/handlers.rs

Description: The on_chain_event_handler subsystem is responsible for parsing and processing onchain transactions. The parse_transaction(...) function is responsible for parsing the transactions and firing an event to be handled by the handle_on_chain_outcome(...) function. The issue is with OLP deposits, which involves a direct transfer(...) call on the USDC contract as shown below in the on_chain_tx_worker process:

```
// on_chain_tx_worker calls.rs
pub(crate) async fn dispatch_olp_deposit_call<P: Provider<AnyNetwork>>(...) -> ... {
    let res = if has_expired {
        Err(Error::bad_request("call expired".into()))
    } else {
        let ResultOrCancel::Result(res) = instrument_async(
            info_span!(
                "retry_in_dispatch_olp_deposit_call",
                pool = deposit.pool_address.encode_hex_with_prefix(),
                amount = deposit.amount.to_string(),
            ),
            retry_with_cancel_context(
                RETRY_CONTEXT,
                aggressive_exponential_backoff,
            ) || async {
                let call = contract.transfer( // @audit USDC transfer
                    deposit.pool_address,
                    usdc_to_base_units_rounded_up(deposit.amount)?,
                );
            }
        );
    }
}
```

In the on_chain_event_handler subsystem, the parse_transaction(...) function identifies the USDC transfers and delegates to handlers, but it is gated behind a conditional check that requires the faucet feature flag is enabled as shown below:

```
pub(crate) async fn parse_transaction(...) -> ... {
    // ...
    match self.cursor_type {
        // ...
        BlockCursorType::EventHandlerHighPriority => {
            if tx_to == self.oracle_address {
                // ...
            } else if cfg!(feature = "faucet")
                && tx_to == self.usdc_address
                && ORACLE_PROVIDER_ADDRESSES.contains(&tx.from())
            {
                #[allow(clippy::single_match)]
                match TestUSDCCalls::abi_decode_validate(tx.input()) {
                    Ok(TestUSDCCalls::transfer(call)) => {
                        return Ok(parse_receipt(&receipt_fut.await?, call).map(Into::into));
                    }
                    _ => {}
                }
            }
        }
    }
}
```

In production environments, as the faucet feature is disabled in Cargo.toml and there is no alternative parsing branch configured to identify USDC transfers. As a result, when an OLP deposit occurs, the parser fails to recognise the transaction and returns Ok(None). Because handle_on_chain_outcome(...) is only invoked when parse_transaction(...) returns a ParsedTransaction, the handle_olp_deposit(...) function remains entirely unreachable in production. This causes OLP deposits to be silently ignored by the system leading to failure in tracking transfers, updating database records, and maintaining accurate OLP deposits.

Recommendation(s): Consider implementing a dedicated parsing branch to handle OLP deposits.

Status: Unresolved

Update from the client:

6.6 [Medium] Failed fee collection batches mark transfers as claimed

File(s): crates/crons/src/on_chain_event_handler/handlers.rs

Description: The `handle_collect_fees(...)` function processes the execution outcomes of on-chain fee collections, handling successes, partial failures, or complete reverts. When a batch fails or partially fails, it extracts the failed batch IDs and updates the database state by calling `mark_fee_collection_batches` with the status set to `FeeCollectionStatus::Failed`.

```
async fn handle_collect_fees(
    tx: &AnyTransaction,
    outcome: &Outcome<FeeBatchProcessed>,
    call: &collectFeesCall,
    db_client: &impl DbClient,
) -> Result<()> {
    // ...
    Some(partial_failures) => {
        let batch_ids_to_fail = partial_failures
            .iter()
            .map(|request| request.requestId)
            .map(Uuid::from_u128)
            .collect::<Vec<Uuid>>();

        try_join!(
            db_client.mark_fee_collection_batches(&batch_ids_to_update, status),
            // @audit Failed batches are explicitly passed here with a Failed status
            db_client.mark_fee_collection_batches(&batch_ids_to_fail, FeeCollectionStatus::Failed)
        )?;
    }
}
```

However, the underlying SQL query implemented in the database module handles transfer updates incorrectly during a batch failure.

While the `fee_collection_batches` table correctly consumes the dynamic status parameter (\$2) to update the batch record to Failed, the secondary UPDATE statement unconditionally updates all underlying transfers table to a hardcoded 'claimed' status'.

```
WITH confirmed_fee_collections AS (
    UPDATE fee_collection_batches
    SET status = $2, -- @audit Properly updates batch status to 'failed' via $2
        claimed_at = CASE WHEN status = 'claimed'::fee_collection_status THEN now() ELSE claimed_at END
    WHERE
        id = ANY($1)
        AND status = 'pending'::fee_collection_status
        AND created_at > NOW() - interval '24 hours'
    RETURNING id, transfer_ids
)
UPDATE transfers
-- @audit-issue Hardcoded status unconditionally marks transfers as 'claimed' even if the batch failed
SET fee_status = 'claimed'::fee_collection_status
WHERE id IN (SELECT unnest(transfer_ids) FROM confirmed_fee_collections)
AND created_at > NOW() - interval '24 hours'
```

When a fee collection batch fails on-chain, its status is correctly marked as failed in the `fee_collection_batches` table, but all of its individual underlying transfers are erroneously marked as claimed in the `transfers` table.

This creates a silent data corruption and an accounting desync where the system considers the protocol fees successfully claimed.

Recommendation(s): Consider modifying the secondary UPDATE query inside the SQL block to dynamically apply the status passed via the \$2 parameter rather than using a hardcoded 'claimed' value.

Status: Unresolved

Update from the client:

6.7 [Medium] IPv6 clients can bypass per-IP rate limiting

File(s): crates/omni/src/http/rate_limit.rs, crates/omni/src/http/mod.rs

Description: The Omni API applies per-IP rate limits to its public auth endpoints (including /auth/generate_signing_data and /auth/issue_jwt) to restrict how many requests a single client can make within a time window. The origin IP is extracted from the cf-connecting-ip header, which Cloudflare populates with the client's real address for both IPv4 and IPv6 connections.

In rate_limit.rs, the check_with_window(...) function builds the rate-limit key by converting the extracted IpAddr directly to a string and concatenating it with the request type:

```
// @audit-issue Rate-limit key uses the full /128 address with no prefix normalization.
let key = format!("{request_type}:{identifier}");
```

For an IPv4 client this key encodes a single address from a typically scarce pool, so the limit is meaningful. For an IPv6 client the key encodes a full /128 address. Cloud providers and residential ISPs routinely route an entire /64 prefix (2^{64} addresses) to a single subscriber. Every address within that prefix produces a distinct key, and therefore a distinct rate-limit bucket, a distinct ban record, and a distinct entry in the OMNI_MAX_ADDRESSES_PER_IP wallet cap enforced by ensure_addresses_per_ip_limit(...).

An attacker who controls a single routed IPv6 /64 (available from most VPS providers for the cost of one server) can bypass all per-IP controls, as follows:

1. Send a request to a public auth endpoint sourced from address <prefix>::1. The request succeeds and one unit is consumed from that address's rate-limit bucket;
2. Send the next request sourced from <prefix>::2. A fresh bucket is created, so the count recorded for <prefix>::1 is irrelevant;
3. Repeat with a new address from the prefix for every request. With 2^{64} available addresses, the per-IP limit and the ban mechanism are never triggered regardless of total request volume;

This was confirmed against the staging environment: rotating the source address across a single /64 allowed significantly more than RATE_LIMIT_MAX_PUBLIC requests to /auth/generate_signing_data with zero 429 responses, while the same request volume from a single IPv4 address (and from a single IPv6 /128) was rate limited as expected.

Recommendation(s): Consider normalizing IPv6 source addresses to their routing prefix before constructing the rate-limit key, masking to a /64 as a reasonable default (a /48 or /56 mask may be more appropriate depending on the threat model), while continuing to key IPv4 addresses at /32. Apply the same normalization consistently to the ban manager key and to the OMNI_MAX_ADDRESSES_PER_IP check so all per-IP controls share one notion of client identity. Alternatively, if IPv6 connectivity is not required for the service, consider disabling IPv6 support on Cloudflare so that all clients are funneled through the scarcer IPv4 address space.

Status: Unresolved

Update from the client:

6.8 [Medium] Missing authentication and rate limits on the /quote WS endpoints

File(s): crates/omni/src/ws/mod.rs

Description: The platform utilizes WebSocket endpoints to stream live market quotes to users. Standard protected endpoints enforce authentication checks, restrict incoming requests per IP address, and track open connections to safeguard system resources. However, the quotes endpoints does not implement these architectural protections. It routes connections directly to the handler functions without requiring a user login, enforcing rate limits, or monitoring connection counts.

Both simple and indicative endpoints are not protected.

```
// ...
"/quotes/simple" => {
    // @audit-issue This route processes requests directly without security wrappers.
    quotes_simple_handler(&mut ws_stream, olp_client.clone(), asset_metadata.clone()).await;
}
// ...
```

```
"/quotes/indicative" => {
    Box::pin(quotes_indicative_handler(
        &mut ws_stream,
        olp_client.clone(),
        db_client,
        pricer_client,
        other_redis_pool,
        asset_metadata.clone(),
        cache_manager,
    ))
    .await;
}
```

This lack of restrictions allows an attacker to open multiple connections for instruments. Because every unique instrument requested forces the application to spawn an independent background polling task, an attacker can load the internal quoter service, making it sluggish or bring down causing a denial of service (DoS).

Recommendation(s): Consider implementing the IP rate limiting for the two endpoints.

Status: Unresolved

Update from the client:

6.9 [Low] Authentication flow trusts Redis records, letting session JWTs be minted for any address

File(s): crates/omni/src/http/auth.rs, crates/omni/src/redis_client.rs

Description: The Omni HTTP API authenticates wallet owners with a Sign-In-With-Ethereum (SIWE) challenge-response. The `generate_signing_data(...)` function builds a SIWE message for a requested address, stores it in Redis under a key derived from that address, and returns it to the caller. The caller signs the message with their wallet and submits the signature to the `issue_jwt(...)` function, which validates it and returns a 7-day session JWT. A second, optional flow lets a user move a session to another device: the `issue_transfer_token(...)` function writes a `TransferTokenMetadata` record to Redis, and the `redeem_transfer_token(...)` function later reads that record back and mints a JWT from it.

Signature checking happens in the `validate_siwe(...)` function. It loads the stored message from Redis and verifies the submitted signature with `verify_eip191(...)`, which recovers the signer's address from the signature and compares it to the address field carried *inside the message*. The message is whatever is currently stored in Redis, and nothing checks that this inner address equals the address whose key was used for the lookup:

```
async fn validate_siwe(address: Address, signature: &[u8; 65], ...) -> AuthResult<Uuid> {
    let eip_message: Message = redis_client
        .fetch_typed_data_for_address(address).await?
        // ...
    // @audit-issue The expected signer is `eip_message.address`, loaded from Redis.
    // @audit-issue Nothing checks that it equals the requested `address`.
    eip_message
        .verify_eip191(signature)
        // ...
}
```

The `issue_jwt(...)` function then issues the JWT for the request's address, resolving the company server-side from the database, so the recovered signer never has to match the address being authenticated:

```
let request_id =
    validate_siwe(body.address, &body.signed_message, redis_client.as_ref()).await?;
// ...
let token = generate_jwt(
    address_rec.company, body.address, request_id, create_default_jwt_scope(), None,
);
```

The transfer flow has the same trust problem in a more direct form. The `redeem_transfer_token(...)` function reads the metadata record from Redis and mints a JWT from its address, company, session_id, and scope fields. The only gate is that the SHA-256 of the submitted secret equals the `secret_sha256` field stored in the record; no signature is involved:

```
let mut metadata = redis_client
    .get_auth_transfer_token_metadata(&body.token.id).await?
    // ...
// @audit-issue The `address`, `company`, `session_id` and `scope` all come from Redis.
// @audit-issue Only the secret hash is checked against caller-supplied input.
if metadata.secret_sha256 != body.token.get_secret_sha256() {
    // ...
}
// ...
let token = generate_jwt(
    metadata.company, metadata.address, metadata.session_id, metadata.scope, None,
);
```

An empty scope makes this worse, because the system treats an empty scope as full access:

```
// @audit An empty `scope` is treated as permitting everything.
fn create_default_jwt_scope() -> Vec<String> {
    Vec::new()
}
```

Both records are stored with no integrity protection, under deterministic and attacker-predictable keys. The SIWE message is persisted as its plaintext text form:

```
async fn save_typed_data_for_address(...) -> OmniApiResponse<()> {  
    let key = create_typed_data_key(address);  
    // @audit-issue Stored as plaintext text with no signature or MAC.  
    let value = eip_message.to_string();  
    // ...  
}
```

And the key is derived solely from the public address:

```
fn create_typed_data_key(address: Address) -> String {  
    format!("{TYPED_DATA_REDIS_KEY}:{address:#x}")  
}
```

Because these records are unauthenticated and predictably keyed, any party that can write to the Redis store can forge them. In the current deployment the Redis instances require no authentication and are network-reachable from beyond the application tier, so this write access is attainable by a network-adjacent attacker rather than only by an already-privileged operator.

Concretely, an attacker who can write to Redis can take over any victim address *V* without ever knowing *V*'s wallet key. Using the SIWE flow:

1. The attacker generates their own wallet key pair (*k*, *A*), where *A* is an address they control;
2. They build a SIWE message whose inner address field is their own address *A*, with timestamps that make it currently valid, and sign it with *k*;
3. They write that signed message into Redis under the key for the victim address *V*. The key is derived solely from *V*, so the attacker can compute it without any secret;
4. They call the `issue_jwt(...)` function with *V* as the request address and their signature from step 2. The `validate_siwe(...)` function loads the attacker's message from *V*'s key, and `verify_eip191(...)` succeeds because the signature matches the attacker-chosen inner address *A*;
5. The handler mints a session JWT for *V*. The attacker now holds a valid session for the victim;

The auth-transfer flow gives an even more direct path, and additionally lets the attacker choose the session's privileges:

1. The attacker picks any secret value and computes its SHA-256;
2. They write a `TransferTokenMetadata` record into Redis with address set to *V*, an empty scope, and `secret_sha256` set to the hash from step 1;
3. They call the `redeem_transfer_token(...)` function with the matching secret. Every check passes, because the attacker controls both the stored record and the submitted secret;
4. The handler mints a session JWT for *V* with full access (empty scope) and with a company of the attacker's choosing;

In both cases the result is a full account takeover of an arbitrary user, without access to that user's wallet.

Recommendation(s): Consider binding the verified SIWE message to the address used to look it up, so a signature can only authenticate the address it was issued for. For the transfer flow, consider making the stored metadata tamper-evident, for example by authenticating it with a server-side secret before storing it and verifying it on read.

Status: Unresolved

Update from the client:

6.10 [Low] Cloudflare-only ALB restriction is bypassable by other tenants

File(s): terraform/aws/modules/networking/main.tf

Description: The production, stage, and testnet environments place the public Application Load Balancer behind Cloudflare and are configured with `cloudflare_only = true` (see `environments/prod.tfvars`, `stage.tfvars`, and `testnet.tfvars`). When this flag is set, the networking module fetches Cloudflare's published IP ranges at apply time and builds the ALB security group so that inbound HTTPS is permitted only from those ranges:

```
data "http" "cloudflare_ipv4" {
  count = var.cloudflare_only ? 1 : 0
  url   = "https://www.cloudflare.com/ips-v4"
}
// ...
dynamic "ingress" {
  // @audit-issue Trusts all of Cloudflare's shared egress ranges, not just this tenant.
  for_each = var.cloudflare_only ? local.cloudflare_ipv4_cidrs : []
  content {
    description = "HTTPS from Cloudflare IPv4"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = [ingress.value]
  }
}
```

The intent is that only requests that have transited Variational's Cloudflare configuration (WAF, rate limiting, bot management, etc.) can reach the origin. However, the allowlist is built from Cloudflare's globally shared egress ranges, which are common to every Cloudflare customer. The security group therefore proves only that a connection originated somewhere inside Cloudflare's network, not that it originated from Variational's Cloudflare account. There is no Authenticated Origin Pulls (mTLS) configuration, no custom origin certificate, and no shared secret header or host-header validation at the origin to distinguish Variational's own Cloudflare zone from any other tenant's.

This is the publicly documented "using Cloudflare to bypass Cloudflare" technique. An attacker who learns the origin's public address (the internet-facing ALB DNS name or one of its public IPs) can route traffic through their own Cloudflare account and reach the origin from a trusted Cloudflare source IP, bypassing every Cloudflare-side protection:

1. The attacker obtains the origin address. The ALB is internet-facing (`internal = false`) with `ip_address_type = "dualstack"`, so it has public IPs and a public AWS DNS name. These are routinely recoverable from historical/passive DNS datasets, certificate transparency logs, SAN entries on the ACM certificate, or other origin-IP discovery techniques;
2. The attacker creates a Cloudflare account, adds a zone for a domain they control, and creates a proxied (orange-cloud) DNS record whose target is Variational's origin address;
3. The attacker disables WAF, rate limiting, and all other protections on their own zone;
4. The attacker sends requests to their domain. Cloudflare proxies them to the origin from Cloudflare's shared egress ranges, which the ALB security group accepts;

The result is that the Cloudflare layer is effectively optional for an attacker: WAF rules, rate limits, bot/DDoS mitigations, and any IP/geo or access policies enforced at Cloudflare can all be skipped, exposing the backend directly to unfiltered attacker traffic. (The Tailscale security-group ingress also permitted by the same block is a separate, scoped path and not affected by this.)

Recommendation(s): Consider enforcing at the origin that requests come from Variational's own Cloudflare account, for example via Authenticated Origin Pulls with a per-zone custom certificate (the default shared Cloudflare certificate is also bypassable) and/or a high-entropy secret header injected by Cloudflare and validated before serving. Cloudflare Aegis (dedicated egress IPs) can further narrow the allowlist. Additionally validate the expected `Host` header at the origin and treat the origin IP/DNS as sensitive.

Status: Unresolved

Update from the client:

6.11 [Low] Incomplete is_success() coverage logs successful transactions as failed

File(s): crates/crons/src/on_chain_event_handler/mod.rs

Description: The `handle_parsed_transaction(...)` function manages the lifecycle of on-chain transactions by executing two sequential operations:

- Triggering core business logic for the transaction via `handle_on_chain_outcome(...)`;
- Tracking and recording the final outcome in the `on_chain_pending_transaction_hashes` database table via `update_pending_transaction_status(...)`;

A structural divergence exists between these two operations. While the core business logic handler exhaustively matches and processes all 10 transaction types, the status-tracking helper method `is_success()` only explicitly accounts for 6 of them.

```
async fn handle_parsed_transaction(...) {
    // 1. Business logic: Exhaustively handles all 10 variants.
    let handle_res = retry(...) || handle_on_chain_outcome(...).await;

    // 2. Transaction status tracking: Relies on the incomplete is_success() helper.
    match handle_res {
        Ok(()) => {
            db_client.update_pending_transaction_status(
                tx.tx_hash(),
                if parsed_tx.is_success() { Success } else { Failure }, // @audit-issue Incomplete coverage defaults to
                Failure
            ).await;
        }
        // ...
    }
}
```

The issue stems directly from the pattern matching implementation inside `ParsedTransaction::is_success()`.

```
impl ParsedTransaction {
    #[must_use]
    pub fn is_success(&self) -> bool {
        matches!(
            self.call,
            ParsedCall::PoolCreation { outcome: Outcome::SuccessWithEvent(_, ..) } |
            ParsedCall::Deposit { outcome: Outcome::SuccessWithEvent(_, ..) } |
            ParsedCall::DepositAtomic { outcome: Outcome::SuccessWithEvent(_, ..) } |
            ParsedCall::Withdrawal { outcome: Outcome::SuccessWithEvent(_, ..) } |
            ParsedCall::FeeCollection { outcome: Outcome::SuccessWithEvent(_, ..) } |
            ParsedCall::OtherPartyAddition { outcome: Outcome::SuccessWithEvent(_, ..) }

            // @audit-issue The macro implicitly falls through to false for the 4 variants below:
            // ParsedCall::OLPToPoolTransfer
            // ParsedCall::OLPBatchDeposit
            // ParsedCall::TreasuryManagementWithdrawals
            // ParsedCall::USDCTransfer
        )
    }
}
```

Because the `matches!` macro doesn't explicitly evaluate `OLPToPoolTransfer`, `OLPBatchDeposit`, `TreasuryManagementWithdrawals`, or `USDCTransfer`, it implicitly falls back to false for these variants, even if their inner execution status was completely successful.

As a result, when any of the four unmapped transaction types execute successfully, the protocol's core state modifies correctly, but the off-chain indexing database falsely records the transaction status as a Failure.

Recommendation(s): Update the `is_success()` function to explicitly cover all 10 variants of the `ParsedCall` enum, ensuring successful executions are accurately reflected in the tracking database.

Status: Unresolved

Update from the client:

6.12 [Low] Missing rate limits on unauthenticated endpoints that use pricing service

File(s): crates/omni/src/http/funding.rs, crates/omni/src/http/metadata.rs, crates/omni/src/http/markets.rs

Description: The Omni API exposes several public-facing, unauthenticated endpoints to allow users to retrieve funding info and asset metadata. For example, endpoints like POST /funding and GET /metadata/supported_assets perform internal operations, such as making synchronous HTTP calls to the internal pricing service or querying the database read replica.

This allows an external unauthenticated attacker to execute a low-cost denial-of-service (DoS) attack that degrades internal services. Because the pricing service is a shared dependency across core features like positions, portfolio, margin calculations, and order pricing, its degradation impacts the entire platform's availability. Furthermore, flooding the /open_interest endpoints with distinct instruments bypasses the 10-second in-memory TTL cache, causing sustained database read replica pressure with zero IP visibility or attribution.

Similar pattern exists for markets endpoints too. The function `fetch_sparklines_for_instruments` fires one pricing request per instrument concurrently via `join_all` to fetch seven days of hourly candles. While the request is capped by a configuration value for the maximum number of instruments per request, the absence of rate limiting can still stress the shared dependency.

Recommendation(s): Consider updating the unauthenticated routes in both the funding and metadata modules to extract the `CFConnectingIP` header and apply the `IpRateLimiter` middleware similar to what is done in `candles.rs`.

Status: Unresolved

Update from the client:

6.13 [Low] No session revocation and no absolute session lifetime

File(s): crates/omni/src/http/mod.rs, crates/omni/src/http/auth.rs

Description: The `Claims` extractor validates the HS256 signature and checks the its expiration time. After the successful validation the 7-day session is enabled. The session can be refreshed with `refresh_jwt()`, however no re-authentication is being performed. The session therefore can be renewed indefinitely, without any cap on number of refreshments or limit of maximum time session. Moreover, there is also no mechanism to revoke the session. Such design opens possible scenarios where the stolen session does not expire.

Recommendation: Consider enforcing an absolute cap on number of refreshments or add a session revocation functionality.

Status: Unresolved

Update from the client:

6.14 [Low] Potential for double execution if transaction worker crashes

File(s): src/on_chain_tx_worker/worker.rs, src/queue.rs

Description: The onchain transaction worker reads queued pending transactions from a database and assigns worker threads to process them sequentially. To prevent transactions that are already in flight from being fetched again, the worker maintains a volatile in-memory hash set called `pending_transactions`. Within the protocol, the `transferFromOLPToPool(...)` function in the Oracle contract is used by authorized entities holding the `PROVIDER_ROLE` to transfer referral reward funds from the OLP wallet to the designated token pool.

```
pub(crate) async fn dispatch_direct_deposit<...>(...) -> ... {
    // ...
    let call = contract.transferFromOLPToPool(
        from_addr,
        pool_address,
        amount_u256,
        pool_uuid,
        rfq_uuid,
        parent_quote_uuid,
    );
    // ...
}
```

However, the state transition between submitting a transaction to the network and marking its status as processed in the database is not atomic. If the transaction worker process crashes or restarts after a transaction has been successfully submitted and mined onchain, but before its database status is updated, the in-memory `pending_transactions` hash set is completely cleared. Upon restarting, the worker fetches the identical pending database row again. The `NonceFeedbackManager` fetches the updated onchain nonce, which has now incremented to $N+1$, causing the worker to sign and broadcast a completely new, valid transaction with identical calldata.

While other functions use contract-level mapping checks to reject replayed actions, the `transferFromOLPToPool(...)` function contains no unique transfer-tracking identifiers or verification mappings.

- `depositUSDC`, `withdrawUSDC`, `processTreasuryManagementWithdrawals`, and `collectFees` are all guarded by the contract's `transfers_processed[transferUuid]` mapping; a replay reverts;
- `atomicDeposit` is guarded by `atomic_deposits_processed[parentQuoteUuid]`;
- ERC-20 permits are guarded by the permit nonce;

The `transferFromOLPToPool` function has no such guard and is therefore vulnerable to double spending:

```
function transferFromOLPToPool(...) external onlyRole(PROVIDER_ROLE) {
    // ...
    // @audit-issue The function processes transfers without checking whether the operation has already been executed.
    token.transferFrom(olpWallet, pool, amount);
    token.transfer(pool, amount);
    // ...
}
```

This means that a routine server redeployment, crash, upgrade, or any other event that halts and restarts the service can cause the worker to process the same pending row twice, executing a second distinct onchain transaction. This results in a double spend of either referral reward funds or counterparty assets transferred from the OLP when a trade is matched and executed. A similar problem exists in the `dispatch_faucet_wallet_request_call` and `dispatch_olp_deposit_call` functions.

Recommendation(s): Consider implementing replay-protection mechanisms in the relevant functions mentioned above to prevent potential double spends.

Status: Unresolved

Update from the client:

6.15 [Low] Redis is deployed without authentication

File(s): `terraform/aws/modules/elasticache/main.tf`, `terraform/aws/modules/networking/main.tf`, `terraform/aws/main.tf`

Description: The backend uses several managed Redis/Valkey clusters (AWS ElastiCache) as caches and stream stores, provisioned with Terraform. These clusters hold security-sensitive material (including auth-transfer tokens, one-time login codes, and SIWE challenge messages) alongside cached account and market data. They are intended to be reached only by the application's ECS tasks running inside the private subnets.

Redis performs no authentication by default: any client that can open a TCP connection to port 6379 may issue commands unless an authentication token is configured. The node-based clusters are created without an authentication token and without transit encryption, so connections are plaintext and unauthenticated:

```
resource "aws_elasticache_replication_group" "redis" {
    replication_group_id = "${var.environment}-redis-ha"
    // ...
    // @audit-issue No `transit_encryption_enabled` is set, so traffic is plaintext.
    // @audit-issue No `auth_token` is set, so the cluster requires no authentication.
    engine_version      = "7.1"
    // ...
}
```

This posture is reflected in the connection URL the application is given, which uses the plaintext `redis://` scheme rather than the TLS `rediss://` scheme. The same pattern (no `auth_token`, no `transit_encryption_enabled`, `redis://` URL) is repeated for the other node-based clusters provisioned by the market-data and stream modules.

The security group fronting Redis does not restrict ingress to the application's task security group. In addition to the ECS task group, it opens port 6379 to the entire VPC CIDR, and (in the prod and stage environments) to a peered partner-account CIDR:

```
resource "aws_security_group" "redis_sg" {
    // ...
    ingress {
        from_port      = 6379
        to_port        = 6379
        protocol        = "tcp"
        security_groups = [aws_security_group.ecs_tasks_sg.id, ...]
    }
    // @audit-issue Ingress is also opened to the entire VPC CIDR, not just the app tier.
    ingress {
        from_port      = 6379
        to_port        = 6379
        protocol        = "tcp"
        cidr_blocks     = [var.vpc_cidr]
    }
    // ...
}
```

The VPC CIDR is reachable from outside the cloud account as well, because a Tailscale subnet router advertises the whole VPC CIDR into the tailnet:

```
module "tailscale" {
  // ...
  // @audit-issue The entire VPC CIDR is advertised into the tailnet as a route.
  advertise_routes = [module.networking.vpc_cidr_block, ...]
  // ...
}
```

As a result, any party that can reach port 6379 from one of these network positions (a compromised or rogue workload elsewhere in the VPC, a host in the peered partner account, or a device on the tailnet) obtains unauthenticated read and write access to every Redis cluster. Such a party can read the stored security-sensitive material, tamper with arbitrary cached values, and issue destructive commands such as flushing a datastore, causing data loss and denial of service. Because traffic is plaintext, a party positioned on any network segment the traffic traverses can also read it passively. Finally, the Tailscale access policy that decides which devices may actually use the advertised route is not present in the repository, so the effective network exposure cannot be verified from the codebase.

Recommendation(s): Consider requiring an authentication token and enabling transit encryption on every Redis/Valkey cluster, and supplying the application with a `rediss://` URL carrying credentials. Consider restricting the Redis security group's ingress to the application task security group only, removing the broad VPC-CIDR and peered-CIDR allowances. Consider enabling encryption at rest. Consider managing the Tailscale access policy under version control so the set of devices able to reach internal services is reviewable, and confirming that the advertised route is not auto-approved for unintended devices.

Status: Unresolved

Update from the client:

6.16 [Low] The `dispatch_direct_deposit(...)` function lacks retry logic and duplicate detection

File(s): `src/on_chain_tx_worker/calls.rs`

Description: The onchain transaction worker is responsible for reading rows from the `on_chain_transaction_queue` table and then assigning a worker to dispatch the onchain call to the specific function on the desired contract. The majority of the dispatch functions in the onchain transaction worker have retry and `send_detect_dupe` logic in place, however the `dispatch_direct_deposit` function is missing these behaviors.

Recommendation(s): Consider wrapping the `transferFromOLPToPool` call with a `retry_cancel_with_context` and deduplication logic to be consistent with all other dispatch functions.

Status: Unresolved

Update from the client:

6.17 [Info] Account row created before referral validation

File(s): `auth.rs, client.rs`

Description: In the function `handle_authenticated_user`, a user's address is passed into a get-or-insert database query that will insert new rows into the `companies` and `addresses` tables if the address does not already have an associated entry. This get-or-insert logic is in `get_or_insert_company_and_address`, which is the first function called when handling an authenticated user:

```
async fn handle_authenticated_user(...) -> ... {
  let address_rec = db.get_or_insert_company_and_address(address).await?;
  if cfg!(not(feature = "stage")) || code.as_ref().is_some_and(|code| code != "BOT") {
    handle_referral_code(db, address, address_rec.company, code).await?;
  }
  // ...
}
```

When a new entry is inserted, the entry is committed to the database. The next check after loading or creating an entry is the referral code validation. If an invalid referral code is passed, or if the `handle_referral_code` function rejects for any other reason, the `?` operator will propagate the error, but the `companies` and `addresses` rows have already been inserted and are not rolled back. As a result, unnecessary rows may be inserted for any wallet with a valid SIWE signature.

Recommendation: Consider validating the referral before performing any persistent database insert. This could be achieved by creating a company UUID before interacting with the database.

Status: Unresolved

Update from the client:

6.18 [Info] Consumed quotes remain available in cache

File(s): crates/omni/src/http/quotes.rs, crates/omni/src/http/orders.rs

Description: Indicative quotes are stored in Redis and exposed through the application cache. When a quote is retrieved for order execution, `fetch_quote_by_address_and_id(...)` performs a read-only fetch:

```
let indicative_quote = redis_client
    .fetch_quote_by_address_and_id(claims.address, body.quote_id)
    .await?;
```

The quote may subsequently be removed from Redis as part of the order flow, but the corresponding cached value is not invalidated. Until the cache entry expires, later requests may continue receiving a quote that is no longer valid as it was already consumed. This breaks the expected invariant that the cache reflects the current state of its underlying Redis entry.

Recommendation(s): Invalidate the cached quote whenever the corresponding Redis value is deleted or consumed. Alternatively, update the retrieval flow so that quote consumption and cache invalidation occur together.

Status: Unresolved

Update from the client:

6.19 [Info] HMAC authentication is not constant-time over key existence

File(s): crates/signing_api/src/auth.rs

Description: `Authenticator::authenticate` looks up the HMAC secret by key UUID and returns early if it is absent, before any HMAC is computed, as shown below:

```
// Unknown UUID: return immediately
let Some(secret) = self.keys.get(&creds.key) else {
    return Err(Error::not_authenticated("invalid request key/signature"));
};
// Known UUID: full HMAC-SHA256 runs
let signature_valid = hmac::verify(...).is_ok();
```

An unknown key UUID returns without running HMAC-SHA256, while a known UUID with a bad signature runs the full computation. Both return the same error body, but the response latency differs. A remote attacker measuring response times can distinguish registered key UUIDs from unregistered ones (an existence oracle). It does not reveal the secret or allow forgery. The key is a random UUID, so the keyspace cannot be enumerated over the network.

Recommendation: Consider running the HMAC verification unconditionally using a dummy secret and return error when the key is absent.

Status: Unresolved

Update from the client:

6.20 [Info] IP rate limiting fails open when client IP is unresolved

File(s): auth.rs, auth.rs, mod.rs

Description: For endpoint functions the `CFConnectingIp` can return `None` when the `cf-connecting-ip` header is absent or unparseable. All IP rate limit checks are gated with an `if let Some(ip)` statement meaning that an IP with a value of `None` will silently skip the rate limit check and the per-IP wallet cap. If this server is reachable without traversing Cloudflare, these IP controls will be ignored since the `cf-connecting-ip` can just be omitted.

```
if let Some(ip) = ip {
    ip_rate_limit.check(ip, "public", *RATE_LIMIT_MAX_PUBLIC).await?;
}
```

Recommendation: Consider failing closed instead of open, by rejecting the traffic if no IP resolves from `CFConnectingIp`. Adding Cloudflare Authenticated Origin Pulls (mTLS) can also help to ensure that requests come from the Cloudflare network.

Status: Unresolved

Update from the client:

6.21 [Info] Identical OLP deposits cannot be reliably distinguished

File(s): crates/crons/src/on_chain_event_handler/verify.rs, crates/crons/src/on_chain_event_handler/handlers.rs

Description: In the `on_chain_event_handler`, each event handler has a corresponding `verify_...` function that matches onchain transaction outcomes with database records. While all of these functions use a unique UUID parameter, such as `transfer_id`, `rfqUuid`, or `poolUuid`, embedded in the onchain calldata to guarantee a one-to-one match, the `verify_olp_deposit(...)` function relies on an address and amount combination that may correspond to multiple records when multiple OLP deposits of the same amount are made to the same pool. The `verify_olp_deposit(...)` function attempts to resolve a record by querying the database for a pending transfer based solely on the `pool_address` and transfer amount:

```
// ...
SELECT t.id
FROM transfers t
  INNER JOIN settlement_pools p ON t.target_pool_location = p.id
WHERE p.address = $1
  AND t.qty = $2
// ...
  AND t.status = 'pending'
```

If multiple identical deposits are enqueued, such as two 1,000 USDC deposits to the same pool, the `verify_olp_deposit(...)` function cannot distinguish between them. It uses `fetch_optional`, which may return an arbitrary record when multiple matches exist. If two identical transfers are queued, the first transaction processed may be paired with either database record. If the second transaction then fails onchain but is matched to the wrong pending record, the system may mark a successful transaction as failed, leave a completed transaction marked as pending, or otherwise update the wrong record.

Recommendation(s): Consider refactoring the OLP deposit mechanism to use a contract method that accepts a `transfer_id` UUID as an onchain parameter.

Status: Unresolved

Update from the client:

6.22 [Info] Missing company validation on WS re-subscriptions leaks pricing data

File(s): /crates/omni/src/ws/mod.rs

Description: The `/quotes/indicative` WebSocket endpoint handles real-time data subscription requests via the `quotes_indicative_handler(...)` function.

When a client establishes an initial connection, the server invokes `wait_for_auth_request(...)` to validate their JSON Web Token (JWT). This process extracts a secure `Claims` struct containing a verified company UUID. This server-signed identifier ensures that downstream pricing engines properly tie quotes to the authorized subscriber.

During the initial subscription setup, the application properly enforces security boundaries. It deserializes the incoming message into a mutable `OmniQuoteRequest` and explicitly overwrites the client-supplied company field with the verified `claims.company` value:

```
// ...
let mut req: OmniQuoteRequest = serde_json::from_str(&msg);
// @audit The client-supplied company is correctly overwritten here.
req.company = Some(claims.company);
process_indicative_subscription(req, ...);
// ...
```

However, a vulnerability exists in the handling of subsequent subscription or re-subscription messages transmitted over the same established connection. The handler deserializes the new message into an immutable `req` struct and passes it directly to the processing logic without enforcing or validating the company identifier:

```
// ...
let req: OmniQuoteRequest = serde_json::from_str(&msg);
// @audit-issue The client-supplied company value passes through without validation.
process_indicative_subscription(req, ...);
// ...
```

As the handler fails to validate that the re-subscription request aligns with the session's authenticated JWT claims, an authenticated user can pass an arbitrary company's UUID in the JSON payload of subsequent messages.

The downstream pricing engine relies entirely on this not validated company field to compute customised, proprietary bid/ask spreads. An attacker can exploit this logic gap to fetch real-time quotes calculated with another firm's confidential commercial spreads, resulting in data leakage and competitive exposure.

Recommendation(s): Modify the re-subscription execution pathway to mirror the initial subscription safety mechanisms. Ensure that the incoming req.company field is strictly overwritten with the session's authenticated claims.company before processing, or explicitly reject requests where the payload's company ID does not match the authenticated session state.

Status: Unresolved

Update from the client:

6.23 [Info] Missing quote UUID zero check allows pulling funds up to max allowance

File(s): src/Oracle.sol, src/SettlementPool.sol

Description: The atomicDeposit(...) function in the Oracle contract allows authorized accounts with the PROVIDER_ROLE to execute token deposits on behalf of users. When processing single-party deposits where partyOneAmountRequested is greater than 0 and partyTwoAmountRequested is 0, the function relies on the atomic_deposits_processed mapping to track and prevent duplicate executions.

However, providing a parentQuoteUuid value of 0 completely bypasses this replay protection due to a conditional state-writing pattern. The function first validates that the transaction has not been processed via require(!atomic_deposits_processed[parentQuoteUuid]). Since uninitialized mapping slots in Solidity default to false, an identifier of 0 successfully passes this check. The contract then executes the token transfers. However, the state update that flags the identifier as used is skipped if parentQuoteUuid == 0. This allows any deposit with a zero parent quote UUID to be repeated any number of times. As long as an allowance has been granted by the involved parties, the repeated deposits will succeed.

```
function atomicDeposit(...) external {
    if (partyOneAmountRequested > 0 && partyTwoAmountRequested == 0) {
        // ...
        // @audit-issue This passes when parentQuoteUuid is 0 because the default value is false.
        require(atomic_deposits_processed[parentQuoteUuid] == false, "this transfer has already been processed");
        // ...
        bool fromSuccess = usdc.transferFrom(partyOneAddress, address(this), partyOneAmountRequested);
        // ...
        // @audit-issue When parentQuoteUuid is 0, this tracking update is skipped entirely.
        if (parentQuoteUuid != 0) {
            atomic_deposits_processed[parentQuoteUuid] = true;
        }
    }
}
```

This is assigned a severity of Info because replaying these deposits would require a compromised or incorrectly behaving provider. Even if this occurred, the funds would be moved from the creator or parties to the settlement pool, from which they could be moved back at a later point. The funds would never leave the custody of the coordinating parties and the settlement pool.

Recommendation(s): Consider adding validation to prevent a parentQuoteUuid of zero from being passed to atomicDeposit(...).

Status: Unresolved

Update from the client:

6.24 [Info] No binding between the authenticated caller and the signing key it may use

File(s): crates/signing_api/src/http.rs, crates/signing_api/src/auth.rs

Description: HMAC authentication verifies that a request was signed with a valid key, but returns no identity. The function authenticate returns Result(<>, Error> and the creds.key is not used for identity purposes either. The signing key is selected entirely by the caller-supplied input.value.key (the KMS alias), which is used both to derive the validation address and as the KMS key_id:

```
auth.authenticate(creds, Some(input.bytes))?; // proves "a valid HMAC key", no identity
let sender = signer.get_address(&input.value.key)...; // caller-chosen alias -> address
validate_transaction(sender, &tx)?; // policy keyed on that address
sign_helper(&input.value.key, ...).await // same alias -> KMS key_id, signs
```

No check ties the authenticated caller to the alias it requests. Any caller holding any valid HMAC secret can name any configured alias, including a privileged one. The service then validates against that key's address and signs with that key. The only limit is validate_transaction, which is keyed on the signing key's address, not the caller, so a caller can obtain signatures for any operation allow-listed for any key. If the HMAC keys are intended to map to distinct privilege levels, this is a broken access control. However, if all HMAC-key holders are equally trusted to drive every key this current implementation is appropriate.

Recommendation: Consider maintaining a mapping of HMAC key (UUID) to permitted aliases (or permitted call types) and enforce it in the handler before signing, so a caller can only use the keys it is authorized for.

Status: Unresolved

Update from the client:

6.25 [Info] No onchain fee segregation for SettlementPools

File(s): src/Oracle.sol, src/SettlementPool.sol

Description: The settlement pools hold all USDC deposited by counterparties as collateral, alongside any accrued fees, as a single ERC-20 balance. As there is no onchain accounting for the fees, the total balance in a pool can be treated as either fees or collateral at the provider's discretion. The PROVIDER role can execute fee withdrawals from any pool, and there are no onchain mechanisms to identify whether an excessive amount of fees has been withdrawn, potentially allowing some or all user collateral to be taken.

Recommendation(s): No change is required. As long as the offchain logic behaves correctly and the provider roles are not compromised, fee withdrawals will behave as expected. A guardrail that tracks the proportion of assets attributable to fees onchain is not viable, as regular updates would incur unsustainable gas costs.

Status: Unresolved

Update from the client:

6.26 [Info] Referral capacity check is a TOCTOU race

File(s): auth.rs, referrals.rs

Description: When handling a referral code in the function `handle_referral_code` the referral is first validated, and then inserted into the database. The validation happens in `validate_referral_code_helper` which, does a capacity check to ensure that the total capacity is not exceeded (`referrals.len() >= total_capacity`). The existing amount of referrals is queried on `self.pg_pool` and then after the validation is done, the new association is entered into the same database at the end of the `handle_referral_code` function.

There is no row lock and no database count $\leq$ capacity constraint, so at the database level it is possible to exceed the capacity. This allows for a time-of-check time-of-use (TOCTOU) race condition where the database query to check the referral capacity may pass, but in the small time window after the check but before the new referral association is written to the database the count may have increased. This could occur with concurrent redemptions of the same code, where they both read the same count and then both insert, over-subscribing beyond the expected capacity.

Recommendation: Consider making the check and insert interactions an atomic transaction rather than breaking it up over two database interactions.

Status: Unresolved

Update from the client:

6.27 [Info] Self role removal via `renounceRole(...)` causes provider counter desync

File(s): src/Oracle.sol, src/TreasuryManagementHelper.sol

Description: The contract `Oracle` manages vital protocol operations by tracking authorized entities through an internal `numProviders` counter. This counter ensures that the system maintains a safe threshold of active participants and prevents the accidental removal of the very last system provider.

However, an alternative entrypoint exists within the inherited `OpenZeppelin AccessControl` structure to remove roles. While the contract explicitly handles updates within its custom `revokeRole(...)` function, it does not account for the `renounceRole(...)` entrypoint. A provider wishing to voluntarily step down can call this function directly to remove their own permissions.

Because the internal tracking counter is not updated when a role is renounced through this alternative execution path, the offchain system state becomes desynchronized from the actual onchain role assignments. A similar problem exists in the `TreasuryManagementHelper` contract which tracks the number of `Oracle` providers.

Recommendation(s): Consider overriding the `renounceRole(...)` function within both the `Oracle` and `TreasuryManagementHelper` contracts.

Status: Unresolved

Update from the client:

6.28 [Info] Signature recovery id is not constrained to 0, 1

File(s): crates/signing_api/src/signer.rs, crates/signing_api/src/http.rs

Description: The recovery ID is derived by trial recovery and returned as the response `v` without a range check:

```
let recovery_id = RecoveryId::trial_recovery_from_prehash(&key_entry.key, prehash, &signature)?;
// ...
v: signature.recovery_id.to_byte(), // Can be 0, 1, 2, or 3
```

The `trial_recovery_from_prehash` searches for `id` in `0..=Self::MAX` with `MAX = 3`, making it possible to return a recovery ID of **2 or 3**. Ethereum's on-chain `ecrecover` precompile rejects those IDs, per the [Ethereum Yellow Paper](#), Appendix F:

> It is assumed that `v` is the 'recovery identifier'. The recovery identifier is a 1 byte value specifying the parity and finiteness of the coordinates of the curve point for which `r` is the x-value; this value is in the range of `[0, 3]`, however we declare the upper two possibilities, representing infinite values, invalid. The value `0` represents an even `y` value and `1` represents an odd `y` value. We declare that an ECDSA signature is invalid unless all the following conditions are true: $(312) \ 0 < r < \text{secp256k1n}$ $(313) \ 0 < s < \text{secp256k1n} \div 2 + 1$ $(314) \ v \in \{0, 1\}$

If a recovery ID of 2 or 3 is produced, the service returns a signature whose `v` the onchain `ecrecover` treats as invalid. It should be noted that this event is incredibly unlikely to happen, it requires a signature's `r` value to be small enough that $r + n < p$ where `n` is the curve order and `p` is the field prime. The odds of this happening in `secp256k1` is approximately one in 2.7×10^{38} .

Recommendation: Consider rejecting x-reduced recovery IDs so the service never returns a non-recoverable `v`:

```
// @audit: Example code
// Recovery ID == 2 or 3
if recovery_id.is_x_reduced() {
    whatever!("signature v not in {{0, 1}}; not recoverable by on-chain ecrecover");
}
```

Status: Unresolved

Update from the client:

6.29 [Info] Terraform CI IAM user has effective admin privileges and escalation paths

File(s): terraform/bootstrap_backend/iam/main.tf

Description: Each environment provisions a dedicated IAM user, `$environment-terraform-user`, whose static access key is handed to GitHub Actions so that CI can run Terraform. The intent of the configuration is least privilege: the permissions are split into separate policies named for compute, security, networking, IAM, and state management. In practice the split is cosmetic, because each policy grants a service wildcard on all resources. The compute policy alone allows full control of EC2, ECS, ECR, load balancing, autoscaling, ACM, ElastiCache, and all of S3:

```
resource "aws_iam_policy" "compute_management_policy" {
  // ...
  Action = [
    "acm:*",
    "autoscaling:*",
    "ec2:*",
    "ecs:*",
    "ecr:*",
    "elasticloadbalancing:*",
    // @audit-issue Grants full access to every S3 bucket in the account, including state.
    "s3:*",
    "elasticache:*"
  ]
  Resource = "*"
  // ...
}
```

The security policy grants unrestricted access to all key management, secrets, and parameter store resources:

```
resource "aws_iam_policy" "security_management_policy" {
  // ...
  Action = [
    // @audit-issue `kms:*` includes `kms:Sign` and `kms:PutKeyPolicy` over every key.
    "kms:*",
    "secretsmanager:*",
    "ssm:*"
  ]
  Resource = "*"
  // ...
}
```

Most significantly, the IAM policy grants write access to IAM across the entire account, including the actions needed to grant new permissions and to pass roles:

```
resource "aws_iam_policy" "iam_management_policy" {
  // ...
  Action = [
    // @audit-issue Create/Put/Attach plus PassRole on all account IAM is a path to admin.
    "iam:Create*",
    "iam:Put*",
    "iam:Attach*",
    "iam:PassRole",
    // ...
  ]
  Resource = "arn:aws:iam::${data.aws_caller_identity.current.account_id}:*"
  // ...
}
```

Because the user can create policies and policy versions, attach managed policies, and create access keys for any principal in the account, it can escalate itself to full administrator regardless of the apparent segmentation, for example by attaching the AWS-managed AdministratorAccess policy to itself, by publishing a new default version of one of its own policies that allows all actions on all resources, or by minting access keys for an existing privileged role. This means anyone who obtains this user's access key (through a leaked CI log, a compromised runner, a workflow run they can influence, or direct misuse) gains full control of the AWS account, with the following concrete consequences:

1. Read every stored credential in the account via `secretsmanager:GetSecretValue` and the `ssm:*` parameter store actions, including any HMAC, database, and third-party secrets kept there;
2. Sign arbitrary payloads with the platform's signing keys: `kms:*` permits rewriting a key policy with `kms:PutKeyPolicy` and then calling `kms:Sign`, bypassing any application-layer validation that normally sits in front of those keys, and also permits destroying keys with `kms:ScheduleKeyDeletion`;
3. Push backdoored container images with `ecr:*` and deploy them by updating ECS services and task definitions with `ecs:*`;
4. Read and corrupt Terraform state in S3, which stores resource attributes (including provider tokens and generated secrets) in plaintext;
5. Create hidden backdoor users, roles, and access keys for persistence that survives rotation of the originally exposed key;

The `$environment-ecs-deployer` user defined in the same file exhibits the same over-broad pattern: it is granted `ecs:*`, `ecr:*`, `secretsmanager:GetSecretValue`, and `iam:PassRole` on all resources, so a leak of its key also yields broad deployment control and the ability to read every secret in the account.

Recommendation(s): Consider applying the principle of least privilege (PoLP) to this account, so that it only has the access needed to complete its tasks, rather than wildcard access that could potentially be abused.

Status: Unresolved

Update from the client:

6.30 [Info] The `claimed_at` timestamp is never updated due to conflicting SQL logic

File(s): `/src/on_chain_event_handler/handlers.rs`

Description: During the on-chain fee collection workflow, the `handle_collect_fees(...)` function invokes `impl_mark_fee_collection_batches(...)` to update the state of fee collection batches in the database.

However, due to a logical contradiction in the SQL UPDATE statement, the `claimed_at` timestamp is never recorded when a batch is successfully claimed.

```
pub(crate) async fn impl_mark_fee_collection_batches(
  // ...
) {
  // ...
  UPDATE fee_collection_batches
    SET status = $2,
        -- @audit-issue 'status' references the pre-update row value, which is always 'pending'
        claimed_at = CASE WHEN status = 'claimed'::fee_collection_status
                          THEN now()
                          ELSE claimed_at
                        END
    WHERE id = ANY($1)
        -- @audit The WHERE clause strictly filters for 'pending' rows
        AND status = 'pending'::fee_collection_status
        AND created_at > NOW() - interval '24 hours'
  // ...
}
```

In PostgreSQL, column references inside the SET clause (such as WHEN status = 'claimed'...) evaluate to the existing value of the row before the update is applied. Because the WHERE clause strictly filters for rows where status = 'pending', the condition WHEN status = 'claimed' will always evaluate to false.

As a result, the query fallback path (ELSE claimed_at) is always executed, leaving the claimed_at column permanently set to its initial NULL value.

Recommendation(s): Modify the CASE WHEN expression to evaluate the incoming status parameter (\$2) instead of the pre-update column state (status)

Status: Unresolved

Update from the client:

6.31 [Info] grantRole(...) cannot administer the PROVIDER_ROLE correctly

File(s): src/Oracle.sol

Description: The function grantRole(...) from the Oracle contract is overridden with additional logic if a specific role is passed. When an administrative account grants the PROVIDER_ROLE to a user, the function is intended to track the total number of providers by incrementing the numProviders counter. In the overridden function, the role is first granted by calling super.grantRole(...). If the role granted was for PROVIDER_ROLE, then there is an explicit flow which ensures the given account doesn't already have the role, and then increments the counter. The purpose of this check is to ensure that an account that already holds the role cannot be granted it again, because otherwise it would increase the counter while the number of accounts holding the role actually stays the same. The overridden function is shown below:

```
function grantRole(bytes32 role, address account) public override whenNotPaused {
    super.grantRole(role, account);

    if (role == PROVIDER_ROLE) {
        // @audit-issue This check always reverts because the role was just granted above.
        require(!hasRole(PROVIDER_ROLE, account), "Oracle: Provider already added.");
        numProviders++;
        emit ProviderAdded(account);
    }
}
```

However, since the super.grantRole(...) call is done before the hasRole(...) check, by the time this if branch is reached, the account already has been given the role. This will in turn cause the require statement to fail. As a result, any attempt to grant the PROVIDER_ROLE to an account through this function will revert. It should be noted that the function addProvider can be used as an alternative flow to add this role to an account, so administration of this role is still possible.

Recommendation(s): Consider executing the role check before granting the role in the grantRole function.

Status: Unresolved

Update from the client:

6.32 [Info] post_limit_order(...) skips validation and accepts non-positive qty

File(s): crates/omni/src/http/orders.rs

Description: The limit / GTC order endpoint receives a GtcOrderRequest through the post_limit_order(...) handler, which delegates to gtc_order_helper(...). The GtcOrderRequest type provides a Validate implementation that rejects a non-positive qty and bounds the slippage fields to [0, 1]:

```
impl Validate for GtcOrderRequest {
    fn validate(&self) -> Result<(), ValidationErrors> {
        // ...
        if let Some(qty) = self.qty
            && let Err(validation_error) = validate_positive_db_decimal(&qty)
        {
            errors.add("qty", validation_error);
        }
        // ...
    }
}
```

Implementing Validate only defines a validate() method; nothing in the framework calls it automatically. The market order path calls body.validate()? before processing the request, but the limit / GTC path never invokes request.validate(), so this validation is dead code on this path. As a result gtc_order_helper(...) consumes the request qty directly, via resolve_order_quantity(...), which returns the provided value unchanged for fixed size orders:

```

async fn gtc_order_helper(request: GtcOrderRequest, /* ... */) -> OmniApiResponse<...> {
  // ...
  // @audit-issue The request reaches this point unvalidated, so a non-positive `qty` is accepted.
  let actual_qty =
    resolve_order_quantity(&instrument_ctx, request.qty, request.is_auto_resize);
  // ...
}

```

Throughout the system qty is treated as an unsigned magnitude, with trade direction derived from side. A non-positive qty breaks that invariant: it is forwarded to the quoter, and would otherwise reach the order submission worker's pre-trade risk checks, which gate on a signed qty * mark_price notional and could be defeated by a negative value (a negative notional never exceeds a > limit threshold).

This is not currently exploitable, because the value is rejected downstream before any order is persisted. The order's RFQ row is inserted inside the order write transaction (insert primary RFQ), and the rfqs table enforces a database constraint that the quantity must be strictly positive:

```

CONSTRAINT "rfqs_qty_check" CHECK (("qty" > (0)::numeric))

```

Inserting the RFQ with a non-positive qty violates this constraint, the insert fails, the transaction rolls back, and the endpoint returns a 500 having persisted nothing, so the risk checks are never reached. The concern is that the endpoint relies on this incidental database backstop (and a separate minimum order notional check) instead of the request validation that was written for exactly this purpose and is currently never executed; the same omission also leaves the slippage bounds unenforced on this path.

Recommendation(s): Consider calling request.validate()? at the start of the limit / GTC order path, mirroring the market order path, so that a non-positive qty and out-of-range slippage are rejected at the API boundary with a clear 4xx error rather than a 500 from a downstream constraint violation. More broadly, consider auditing handlers that define a Validate implementation but receive their body through a plain Json / WithRejection extractor, to confirm validate() is actually invoked.

Status: Unresolved

Update from the client:

6.33 [Info] The function permit_helper does not support Safe or ERC-4337 wallets

File(s): omni/src/http/on_chain.rs

Description: The function permit_helper(...) within the on_chain router handles incoming POST /permit requests to bridge offchain signatures into onchain. This endpoint is responsible for checking signature validity, identity binding, and authorization before executing the underlying permit(...) function on the smart contract via the onchain client. The current verification architecture forces an ECDSA signature recovery routine unconditionally for every signature submission:

```

// ...
// @audit-issue This unconditionally runs ECDSA recovery and will fail for contract wallets.
let signer_addr = body.signature.recover_address_from_prehash(&permit_hash)
  .whatever_msg("recovering address from signature"?);

if signer_addr != permit_message.owner {
  return Err(common::Error::bad_request(
    "the provided signature is not valid for the provided message".into(),
  ));
}
// ...

```

This structure assumes that every signature is a standard 65-byte ECDSA signature produced by an Externally Owned Account (EOA). The alternative ERC-1271 smart contract verification path resides later in the logic inside the verify_smart_contract_signature(...) function, but it is only reached if the initial ECDSA condition passes. Furthermore, the contract signature verification logic specifically checks for an exact match against an EIP-7702 layout:

```

// ...
// inside verify_smart_contract_signature(...)
// @audit-issue This check restricts validation exclusively to EIP-7702 designators.
if code.len() != 23 || code[0..3] != EIP7702_PREFIX {
  return Ok(());
}
// ...

```

As a result, smart contract wallets such as Safe multisigs or ERC-4337 smart accounts fail to execute successfully.

Recommendation(s): If support for Safe or ERC-4337 wallets is desired, consider adjusting the signature verification flow to detect whether the signature field should be checked using ECDSA, or if smart contract code exists then check against EIP-7702, ERC-4337 or ERC-1271.

Status: Unresolved

Update from the client:

6.34 [Info] The price endpoint lacks a total subscription cap per connection

File(s): /crates/omni/src/ws/mod.rs

Description: The /prices Web-socket endpoint allows users to subscribe to real-time price updates for various market instruments. When a user submits a subscription request, the `handle_subscription_request(...)` function processes the incoming request and ensures that the number of instruments requested at once does not exceed the `MAX_INSTRUMENTS_PER_REQUEST` limit. However, the function only validates individual subscription requests. It does not check the total accumulated number of active subscriptions stored in the `price_stream` map for that specific connection.

```
async fn handle_subscription_request(...) -> bool {
    // ...
    // @audit This only validates the number of instruments within a single request.
    if request.instruments.len() > *MAX_INSTRUMENTS_PER_REQUEST {
        // ...
        return false;
    }
    // ...
}
```

After performing the per-request validation, the function processes the subscription actions and inserts the unique instruments into the active stream map.

```
// ...
match request.action {
    SubscriptionAction::Subscribe => {
        for instrument in unique_instruments {
            // ...
            if price_stream.contains_key(&instrument) {
                continue;
            }
            // ...
            let stream = price_publisher.subscribe(instrument.clone());
            // @audit-issue New instruments are inserted without verifying total stream size.
            price_stream.insert(instrument, stream);
        }
    }
    // ...
}
```

An attacker can exploit this behaviour to accumulate an unbounded number of subscriptions on a single connection, leading to excessive resource consumption and potential denial of service (DoS) on the node.

Recommendation(s): Consider tracking the total number of active subscriptions for each connection and enforcing a global cap before inserting new instruments into the `price_stream` map.

Status: Unresolved

Update from the client:

6.35 [Info] Unnecessary USDC double transfers

File(s): Oracle.sol

Description: The Oracle has functions `transferFromOLPToPool` and `atomicDeposit` which transfers USDC from some sender address to another recipient address, using itself as a middleman during the transfer process. This flow can be shown as A -> B -> C with A being the sender, B being the oracle, and C being the recipient. Since A has given an allowance to B to spend on its behalf, B is able to execute a `transferFrom` to directly send funds from A -> C, skipping the intermediary transfer and saving gas costs.

```
function transferFromOLPToPool(...) ... {  
    // ...  
    IERC20 usdc = factory.usdcAddress();  
    bool fromSuccess = usdc.transferFrom(olpWallet, address(this), amount); // A -> B  
    // ...  
    bool toSuccess = usdc.transfer(poolAddress, amount); // B -> C  
    // ...  
}
```

Recommendation(s): Consider using the `transferFrom` to move assets directly to the recipient address in `transferFromOLPToPool` and `atomicDeposit`.

Status: Unresolved

Update from the client:

6.36 [Info] WebSocket rate limit result is ignored in /prices

File(s): omni/src/ws/mod.rs, omni/src/http/mod.rs

Description: The `/prices` WebSocket handler calls `handle_subscription_request(...)` to apply rate limiting to incoming subscription messages. However, the returned result is assigned to `_res` and never evaluated:

```
let _res = handle_subscription_request(...).await;  
  
if price_stream.is_empty() {  
    return;  
}
```

As a result, rate-limited or banned clients are not disconnected and may continue sending messages. Each rejected message still performs Redis operations, and Redis errors or timeouts are treated as `RateLimitCheckResult::Allowed` by `check_ip_rate_limits(...)`, further weakening the rate-limit enforcement.

Recommendation(s): Evaluate the result returned by `handle_subscription_request(...)` and take an appropriate action when the request is rejected, such as terminating the WebSocket connection.

Status: Unresolved

Update from the client:

6.37 [Info] Lazy address parsing can unexpectedly abort the process

File(s): crates/signing_api/src/config.rs, crates/signing_api/src/validate.rs, crates/common/src/lib.rs. Cargo.toml

Description: All address/whitelist configurations the request path uses is held in LazyLock statics whose initializer reads an environment variable and panics on a parse failure:

```
pub static OLP_TRANSFER_WHITELIST: LazyLock<HashSet<Address>> = LazyLock::new(|| {
    std::env::vars()
        .filter(|(k, _)| k.starts_with("OLP_TRANSFER_WHITELIST_"))
        .map(|(k, v)| v.parse:::<Address>().unwrap_or_else(|_| panic!("invalid address in {k}")))
        .collect()
});
```

The same pattern applies to USDC_ADDRESS, ORACLE_ADDRESS, TM_HELPER_ADDRESS, MM_BOT_COMPANY_WALLET_ADDRESS, ORACLE_PROVIDER_ADDRESSES, ORACLE_LOW_PRIORITY_PROVIDER_ADDRESSES, and REFERRAL_REWARD_WALLET_ADDRESS. None of these statics are referenced during startup, they are first dereferenced inside parse_input / validate_transaction while handling a request. The /v1/status health check does not touch them.

A malformed address in any of these environment variables is not detected at startup, the service boots and works for some time, and then panics on the first request that dereferences the bad static. With panic = "abort", that panic terminates the whole process. Because every instance shares the same config, the same request class crash-loops each instance as traffic reaches it.

Recommendation: Consider parsing the values from the environment variables upon process start, rather than using LazyLock.

Status: Unresolved

Update from the client:

6.38 [Best Practices] Decrypted SSM HMAC secret is included in error messages

File(s): crates/signing_api/src/auth.rs

Description: In the production loader (from_parameter_store), when an SSM parameter value is not valid 32-byte hex, the already-decrypted value is interpolated into the error message, which propagates to main and is logged. On a malformed parameter the cleartext key material is written to startup/error logs, defeating the SecureString-at-rest protection. The same pattern exists in the dev-only loaders (auth.rs new_from_env, signer.rs:243 which logs a raw private key), which are not compiled in production.

Recommendation(s): Consider omitting the secret value from the error messages and instead only reference the parameter name and at most the observed length.

Status: Unresolved

Update from the client:

6.39 [Best Practices] Error handling recommendations

File(s): *.sol

Description: The smart contract code has many input validations and checks in place. While the actual checks are thorough, the error messages are passed as a String type in the revert data. This approach can lead to higher gas costs and had caused some error message inconsistencies across the codebase. For example, the Oracle contract has checks across multiple functions to ensure that a UUID is associated with a pool. Even though the check is the same, the error message varies:

```
// All examples are from "Oracle.sol"
require(poolAddress != address(0), "no pool mapping found for given poolUuid");
require(poolAddress != address(0), "Invalid poolUuid");
require(poolAddress != address(0), "Oracle: SettlementPool address cannot be zero");
require(poolAddress != address(0), "Pool not found for the given UUID");
```

Solidity supports a built-in error type which could be used instead, improving gas efficiency, code readability, and maintainability. A reference on custom errors in Solidity can be found [here](#).

Recommendation(s): Consider implementing errors using the Solidity Error type rather than passing errors as a String.

Status: Unresolved

Update from the client:

6.40 [Best Practices] Fragmented fee collection

File(s): src/Oracle.sol

Description: The collectFees(...) function allows a provider to withdraw funds from the settlementPool contract as protocol fees, which can be called only by the provider role. A relevant code snippet is shown below:

```
function _processFeeRequests(...) internal returns (...) {
    // ...
    try SettlementPool(poolAddress).withdrawFees(
        msg.sender, // Caller receives the fees
        requests[i].amountRequested,
        requests[i].feesBatchId
    ) {...} catch Error(string memory reason) {...} catch {...}
}
}
```

Since there can be multiple accounts with the provider role, the fee collection may be carried out by different accounts over time, leading to fragmented fees that would then need to be pooled together at a later point.

Recommendation(s): Consider configuring a fee collection address where all the fees are collected, instead of passing fees to the msg.sender which can cause fee fragmentation.

Status: Unresolved

Update from the client:

6.41 [Best Practices] Lack of role segregation between standard withdrawals and treasury operations

File(s): src/Oracle.sol

Description: The Oracle contract implements access control using PROVIDER_ROLE for multiple distinct functions. However, this single role is shared between routine, high-frequency automated actions and highly sensitive administrative treasury operations.

As an example, withdrawUSDC(...) processes standard individual user withdrawals from specific settlement pools and is gated by PROVIDER_ROLE.

```
// ...
function withdrawUSDC(...) external onlyRole(PROVIDER_ROLE) {
    // @audit Routine user withdrawals are authorized using PROVIDER_ROLE.
    pool.withdrawUSDC(...);
}
}
```

Like wise, the highly privileged administrative batch withdrawal function, processTreasuryManagementWithdrawals(...), is also gated by the exact same PROVIDER_ROLE.

```
// ...
function processTreasuryManagementWithdrawals(...) external onlyRole(PROVIDER_ROLE) {
    // @audit-issue Sensitive batch treasury operations use the same PROVIDER_ROLE.
    for (uint256 i = 0; i < withdrawals.length; i++) {
        // ...
    }
}
}
```

This breaks the principle of least privilege, this shared access model creates a significant security risk. Under the current implementation, if a provider key is compromised, the attacker immediately gains full authorisation to execute batch treasury movements and drain multiple pools across the protocol.

Recommendation(s): Consider establishing distinct roles to segregate protocol operational privileges. Introduce a dedicated role, such as TREASURY_MANAGER_ROLE, exclusively for administrative batch actions, ensuring that a compromise of an automated provider key does not result in a full treasury compromise.

Status: Unresolved

Update from the client:

6.42 [Best Practices] Missing functions in IOracle or parent interfaces

File(s): IOracle.sol

Description: The IOracle interface and its parent interfaces declare all functions that the Oracle contract should implement, however some functions appear to be missing. The following is a list of functions which are present in the Oracle contract but aren't declared in the interface:

```
atomic_deposits_processed(uint128)
processTreasuryManagementDeposits(address, TreasuryManagement.Deposit[])
getPool(uint128)
```

Recommendation(s): Consider adding the listed functions above to IOracle or its parent interfaces. It should be noted that the function getPool is recommended to be removed in another finding. If the function is removed from the Oracle then it should not be added to the interface.

Status: Unresolved

Update from the client:

6.43 [Best Practices] Missing input validation on referral code in set_code(...)

File(s): crates/omni/src/http/referrals.rs

Description: The set_code(...) endpoint allows users to submit a referral code to register their referrer. Under normal operation, valid referral codes are automatically generated by the generate_omni_code(...) function, which produces strings with a total length of exactly 12 characters. However, the SetCodeRequest struct accepts any arbitrary user-supplied string for the code field without enforcing any length limits or character format constraints:

```
#[derive(Deserialize)]
pub struct SetCodeRequest {
    // @audit-issue The code field lacks validation attributes to enforce format constraints.
    pub code: String,
}
```

Because input validation is missing at the API layer, the raw user-supplied string is directly passed into multiple database lookups. The first lookup is performed inside the fetch_referral_code(...) function, which runs a complex aggregation query against the database:

```
SELECT json_build_object(...) AS "code!", COALESCE(rcc.rows, '[]'::json) AS "capacity!"
FROM referral_codes rc
LEFT JOIN addresses a ON a.address = rc.referrer
LEFT JOIN LATERAL (
    SELECT JSON_AGG(...) as rows
    FROM referral_code_capacity rcc
    WHERE rcc.referral_code_id = rc.id
) rcc ON true
// @audit-issue The query executes an expensive lookup using the unvalidated input string.
WHERE rc.code = $1
```

Immediately following the first lookup, a second database query is triggered within the fetch_wallets_associated(...) function using the exact same unvalidated input string:

```
SELECT referee FROM public.referrals r
INNER JOIN public.referral_codes c ON c.id = r.referral_code_id
-- @audit The unvalidated parameter is checked against the database before the transaction.
WHERE c.code = $1
```

This lack of early rejection means that an attacker can submit an arbitrarily large string (e.g., a payload of 1MB or greater) to the endpoint. Although the application utilizes parameterized SQL bindings via sqlx which prevents standard SQL injection vulnerabilities, the database is still forced to receive, allocate memory for, and perform resource-intensive string comparisons across the indexed referral_codes.code column.

Recommendation(s): Consider implementing field-level input validation constraints directly on the SetCodeRequest struct.

Status: Unresolved

Update from the client:

6.44 [Best Practices] The SettlementPoolDeployer contract and interface is unused

File(s): src/SettlementPoolDeployer.sol, src/interfaces/ISettlementPoolDeployer.sol

Description: The protocol utilizes the SettlementPoolFactory contract along with OpenZeppelin's cloned minimal proxies to deploy new settlement pools. When a pool is deployed, configuration parameters are provided directly to the initialize(...) function of the SettlementPool contract. However, the SettlementPoolDeployer contract and its accompanying ISettlementPoolDeployer interface remain unused in the project. The deploy(...) function within SettlementPoolDeployer is marked internal with no external wrappers, the parameters(...) function is never called.

Recommendation(s): Consider removing the SettlementPoolDeployer.sol and ISettlementPoolDeployer.sol files from the repository entirely.

Status: Unresolved

Update from the client:

6.45 [Best Practices] Unused dependencies are left in production contract

File(s): src/SettlementPool.sol

Description: The SettlementPool contract is designed to manage pool settlements and serves as the implementation contract behind EIP-1167 minimal proxies. However, the contract imports the Forge standard library's console logging utility, which is a development and testing dependency. While no console.log() calls are actively executed within the logic, including this import serves no purpose in a production environment.

Recommendation(s): Consider removing the unused console import.

Status: Unresolved

Update from the client:

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Omnis's documentation

The primary source of documentation was docs.variatonal.io, which provides an overview of Variational, the Omni protocol, and the Omni Liquidity Provider (OLP). This documentation helped provide a high-level understanding of the protocol's onchain and offchain logic. For more detailed documentation covering specific logic, inline comments were useful in describing the expected behavior of functions and particularly complex parts of the codebase.

In addition to the documentation, the team provided the Nethermind Security team with access to the testing environment during sync calls. The development team was collaborative throughout the audit, providing answers to all questions raised by the Nethermind Security team during calls and through asynchronous communication.

8 Test Suite Evaluation

8.1 Tests Output

```

test http::candles::tests::four_hour_at_cap_ok ... ok
test http::candles::tests::fencepost_unaligned_endpoints_count_same_as_aligned ... ok
test http::candles::tests::five_minutes_over_cap_rejected ... ok
test http::candles::tests::fifteen_minutes_over_cap_rejected ... ok
test http::candles::tests::one_day_at_cap_ok ... ok
test http::candles::tests::four_hour_over_cap_rejected ... ok
test http::candles::tests::one_day_over_cap_rejected ... ok
test http::candles::tests::one_hour_at_cap_ok ... ok
test http::candles::tests::five_minutes_at_cap_ok ... ok
test http::candles::tests::fifteen_minutes_at_cap_ok ... ok
test http::candles::tests::one_hour_over_cap_rejected ... ok
test http::candles::tests::one_minute_at_cap_ok ... ok
test http::candles::tests::start_after_end_rejected ... ok
test http::candles::tests::one_minute_over_cap_rejected ... ok
test http::candles::tests::over_cap_reports_inclusive_count ... ok
test http::candles::tests::start_equals_end_rejected ... ok
test http::candles::tests::subbucket_span_counts_one_candle ... ok
test http::candles::tests::thirty_minutes_at_cap_ok ... ok
test http::candles::tests::thirty_minutes_over_cap_rejected ... ok
test http::exports::tests::clamp_crossing_gte_rejected ... ok
test http::exports::tests::created_at_gte_must_be_before_lte ... ok
test http::exports::tests::created_at_lte_outside_recency_buffer_unchanged ... ok
test http::exports::tests::created_at_lte_in_future_clamps_to_recency_cutoff ... ok
test http::exports::tests::time_range_exceeds_max_rejected ... ok
test http::exports::tests::created_at_lte_within_recency_buffer_is_clamped ... ok
test http::exports::tests::transfer_types_invalid_for_trades ... ok
test http::exports::tests::transfer_types_required_for_transfers ... ok
test http::funding::tests::parses_v2_funding_query_params ... ok
test http::metadata::tests::parses_v2_open_interest_query_params ... ok
test http::metadata::tests::supported_assets_response_keeps_unfiltered_empty_result ... ok
test http::markets::tests::parses_v2_sparklines_query_params ... ok
test http::metadata::tests::supported_asset_exists_matches_requested_cex_asset ... ok
test http::metadata::tests::supported_asset_exists_matches_requested_dex_asset ... ok
test http::metadata::tests::supported_assets_response_keeps_filtered_match ... ok
test http::orders::tests::stop_limit_validation_tests::test_no_conflicts_when_no_existing_stop_limit_orders ... ok
test http::orders::tests::stop_limit_validation_tests::test_fixed_size_stop_limit_conflicts_with_auto_resize ... ok
test http::orders::tests::stop_limit_validation_tests::test_auto_resize_stop_limit_conflicts_with_fixed_size ... ok
test http::orders::tests::stop_limit_validation_tests::test_no_conflicts_when_same_type_stop_limit_orders ... ok
test http::orders::tests::stop_limit_validation_tests::test_stop_limit_orders_dont_conflict_with_tp_sl ... ok
test http::metadata::tests::supported_assets_response_returns_not_found_for_missing_filtered_cex_asset ... ok
test http::orders::tests::test_child_slippage_limit_defaults_to_none_when_missing ... ok
test http::orders::tests::stop_limit_validation_tests::test_use_mark_price_irrelevant_to_stop_limit_validation ... ok
test http::orders::tests::test_complex_scenario_multiple_conflicts ... ok
test http::orders::tests::test_cross_type_orders_allowed ... ok
test http::leaderboard::tests::leaderboard_v2_requires_authentication ... ok
test http::orders::tests::test_gtc_request_accepts_child_slippage_limits_in_range ... ok
test http::orders::tests::test_gtc_request_rejects_out_of_range_child_slippage_limits ... ok
test http::orders::tests::test_handle_trigger_based_order_validation_logic ... ok
test http::orders::tests::test_no_conflicts_when_no_existing_orders ... ok
test http::orders::tests::test_mixed_tp_and_sl_conflicts ... ok
test http::orders::tests::test_no_conflicts_when_same_type_orders ... ok
test http::orders::tests::test_no_conflicts_with_non_tp_sl_orders ... ok
test http::orders::tests::test_sl_auto_resize_conflicts_with_fixed_size ... ok
test http::orders::tests::test_sl_fixed_size_conflicts_with_auto_resize ... ok
test http::orders::tests::test_standalone_orders_only_validate_their_own_type ... ok
test http::orders::tests::test_stop_limit_orders_ignored_in_tp_sl_validation ... ok
test http::orders::tests::test_tp_auto_resize_conflicts_with_fixed_size ... ok
test http::orders::tests::test_tp_fixed_size_conflicts_with_auto_resize ... ok
test http::orders::tests::test_use_mark_price_irrelevant_to_validation ... ok
test http::quotes::tests::test_fok_request_accepts_child_slippage_limits_in_range ... ok
test http::quotes::tests::test_fok_request_rejects_out_of_range_child_slippage_limits ... ok
test http::quotes::tests::test_quadratic ... ok
test http::quotes::tests::test_sanitize_quote_qty_for_ws_transport_normalizes ... ok
test http::quotes::tests::test_sanitize_quote_qty_for_ws_transport_rejects_large_mantissa ... ok
test http::tests::rejects_ambiguous_underlying_for_perpetual_resolution ... ok
test http::tests::does_not_cross_match_perpetual_variant_with_wrong_instrument_type ... ok
test http::tests::rejects_non_perpetual_instrument_type ... ok

```

```

test http::tests::rejects_unsupported_perpetual_asset ... ok
test http::tests::resolves_supported_perpetual_cfd_for_underlying ... ok
test http::tests::resolves_supported_perpetual_rwa_future_for_underlying ... ok
test http::tests::resolves_supported_perpetual_instrument_for_unique_underlying ... ok
test http::tests::test_check_scope ... ok
test http::tests::validates_supported_perpetual_asset_for_rwa_and_cfd_variants ... ok
test http::tests::validates_supported_perpetual_assets_for_cex_and_dex ... ok
test portfolio_snapshot::tests::test_generations_has_changed ... ok
test portfolio_snapshot::tests::test_cache_miss_for_unknown_company ... ok
test portfolio_snapshot::tests::test_gtc_orders_generation_change_invalidates ... ok
test portfolio_snapshot::tests::test_arc_clone_is_same_data ... ok
test portfolio_snapshot::tests::test_pools_generation_change_invalidates ... ok
test portfolio_snapshot::tests::test_company_isolation ... ok
test portfolio_snapshot::tests::test_trades_generation_change_invalidates ... ok
test portfolio_snapshot::tests::test_cache_lifecycle_hit_then_invalidate ... ok
test redis_client::tests::decode_rejects_empty ... ok
test portfolio_snapshot::tests::test_transfers_generation_change_invalidates ... ok
test redis_client::tests::bulky_fields_are_stripped ... ok
test redis_client::tests::cached_quote_decode_does_not_pretend_to_have_timestamp ... ok
test redis_client::tests::decode_rejects_garbage ... ok
test redis_client::tests::key_prefix_is_versioned ... ok
test redis_client::tests::roundtrip_dated_future ... ok
test redis_client::tests::encoded_size_is_compact ... ok
test redis_client::tests::roundtrip_dex_spot ... ok
test redis_client::tests::roundtrip_perp ... ok
test redis_client::tests::roundtrip_spot ... ok
test ws::connection_logger::tests::test_close_reason_strings ... ok
test ws::connection_logger::tests::test_connection_event_builder ... ok
test redis_client::tests::roundtrip_vanilla_option ... ok
test ws::connection_logger::tests::test_outcome_strings ... ok
test ws::tests::test_combined_dedup_and_skip ... ok
test ws::tests::test_deduplicates_instruments ... ok
test ws::tests::test_skips_already_subscribed ... ok
test ws::tests::test_rejects_too_many_instruments ... ok
test ws::tests::test_accepts_max_instruments ... ok
test http::quotes::tests::test_indicative_quote_olp_failure ... ok
test http::quotes::tests::test_indicative_quote ... ok
test ws::tests::test_ws_rejects_oversized_message ... ok
test ws::tests::test_ws_accepts_valid_message ... ok
test ws::tests::test_ws_rejects_oversized_json_payload ... ok

test result: ok. 107 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.06s

```

```

Ran 4 tests for test/SettlementPool.t.sol:OracleTest
[PASS] testDepositUSDCOnlyParties() (gas: 365463)
[PASS] testDepositUSDCOnlyPartiesUnauthorized() (gas: 279659)
[PASS] testUSDC() (gas: 2348)
[PASS] testWithdrawUSDCOnlyPartiesUnauthorized() (gas: 340452)

Ran 1 test for test/TreasuryManagementHelper.t.sol:TreasuryManagementHelperTest
[PASS] testBatchDeposit() (gas: 669381)

Ran 20 tests for test/Oracle.t.sol:OracleTest
[PASS] testAtomicDeposit() (gas: 332229)
[PASS] testCreatePoolInsufficientAllowance() (gas: 75151)
[PASS] testCreatePoolInvalidAddresses() (gas: 86764)
[PASS] testCreatePoolNoAllowance() (gas: 47504)
[PASS] testCreatePoolPaused() (gas: 30408)
[PASS] testCreatePoolWithFee() (gas: 288345)
[PASS] testDeposit() (gas: 318796)
[PASS] testDepositDuplicateTransferUuid() (gas: 347144)
[PASS] testDepositNoApprove() (gas: 247949)
[PASS] testDepositZeroAmount() (gas: 281677)
[PASS] testGasComparison() (gas: 3524939)
[PASS] testProcessTreasuryManagementDeposits() (gas: 677470)
[PASS] testProcessTreasuryManagementWithdrawals() (gas: 448695)
[PASS] testUSDC() (gas: 2372)
[PASS] testWithdrawExcessAmount() (gas: 351035)
[PASS] testWithdrawFees() (gas: 380447)
[PASS] testWithdrawFeesDuplicateAttempt() (gas: 399597)
[PASS] testWithdrawFeesNewProvider() (gas: 417088)
[PASS] testWithdrawFeesNonProvider() (gas: 353678)
[PASS] testWithdrawFeesTwoPoolsOneRequest() (gas: 690661)

```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.